

VII. APACHE 2.0

VIII. VIRTUAL HOSTING

Setting Up a Web Server

I-Global Section

```

1  ServerType      standalone
2  ServerRoot     /etc/httpd/
3  LockFile       /var/lock/httpd.lock
4  PidFile        /var/run/httpd.pid
5  ScoreBoardFile /var/run/httpd.scoreboard
6  ResourceConfig  conf/srm.conf [deprecated]
7  AccessConfig   conf/access.conf [deprecated]

```

Performance Tuning - Managing Child Processes

```

8  Timeout        300
9  KeepAlive      off
10 MaxKeepAliveRequests 100
11 KeepAliveTimeout 15
12 MinSpareServers 5
13 MaxSpareServers 20
14 StartServers   8
15 MaxClients     150
16 MaxRequestsPerChild 0
17 ThreadPerChild 0
18 ListenBackLog  511
19 Listen         80
20 BindAddress     Deprecated. Use Listen.
21 LoadModule      /etc/httpd/modules/ which is a symlink to /usr/lib/httpd/modules/
22 <IfDefine>....</IfDefine>
23 ClearModuleList  Deprecated
24 AddModule        Deprecated
25 ExtendedStatus   Off.
26 Include conf.d/*.conf  PHP/Perl misc confs

```

II-Main Section

```

27 Port           Deprecated. Use Listen.
28 User apache
29 Group apache
30 ServerAdmin
31 ServerName
32 DocumentRoot   /var/www/html

```

Scoping And Scopes

```

33 <Directory>....</Directory>
34 <DirectoryMatch>....</DirectoryMatch>
35 <Files>....</Files>
36 <FilesMatch>....</FilesMatch>
37 <Location>....</Location>
38 <LocationMatch>....</LocationMatch>
39 Options
40 None
41 All
42 FollowSymLinks
43 ExecCGI
44 Includes
45 IncludesNOEXEC
46 Indexes
47 MultiViews
48 SymLinksIfOwnerMatch

```

Server Security

Restricting Access to sensitive contents Using HTTP Authentication

```

49 AllowOverride
50 All
51 None
52 AuthConfig [All directives below are given in the .htaccess file and not in httpd.conf]
53 AuthName
54 AuthType
55 AuthUserFile
56 AuthGroupFile
57 AuthDBUserFile
58 AuthDBGroupFile
59 AuthDBMUserFile
60 AuthDBMGroupFile
61 require
62 Limit
63 LimitExcept
64 FileInfo

```

65 Indexes

66 Options

Restricting Access to sensitive contents Using IP or Hostname

67 Order

68 allow

69 deny

70 allow from all

71 deny from all

72 mutual failure

73 satisfy

74 AccessFileName .htaccess

75 DirectoryIndex

76 UseCanonicalName Off

77 TypesConfig /etc/mime.types

Content Negotiation Directives

78 AddType

79 DefaultType text/plain

80 DefaultIcon /icons/unknown.gif

81 DefaultLanguage en.en

82 AddLanguage en.en.etc

83 HostNameLookups On/Off/double

84 Redirect

HTTP as a Proxy Server

85 ProxyRequests

86 ProxyRemote

87 ProxyPass

88 ProxyPassReverse

89 ProxyBlock

90 AllowCONNECT

91 ProxyReceiveBufferSize

92 NoProxy

93 ProxyDomain

94 ProxyVia

HTTP as a Caching Server

95 CacheRoot

96 CacheSize

97 CacheGcInterval

98 CacheMaxExpire

99 CacheLastModifiedFactor

100 CacheDefaultExpire

101 NoCache

102 CacheDirLength

103 CacheDirLevels

104 CacheForceCompletion

105 CacheNegotiatedDocs

106 UserDir

107 ServerAlias

Logging /etc/httpd/logs/ which is a symlink to /var/log/httpd/

108 ErrorLog

109 LogLevel

110 LogFormat

111 TransferLog

IX. APACHE

X. VIRTUAL HOSTING

What Is HTTP, and How Does It Work?

- Put simply, HTTP is the protocol that allows Web browsers and servers to communicate. It forms the basis of what a Web server must do to perform its most basic operations.
- HTTP started out as a very simple protocol, and even though it has had numerous enhancements, it is still relatively simple. As with other standard Internet protocols, control information is passed as plain text via a TCP connection. In fact, HTTP connections can actually be made using standard "telnet" commands.
- For example:

```
/home/foo > telnet www.shiva.bom.labs.net 80
GET /index.html HTTP/1.0
<- Extra char return needed
```
- Note that port 80 is the default port a Web server "listens" on for connections.
- In response to this HTTP GET command, the Web server returns to us the page "index.html" across the telnet session, and then closes the connection to signify the end of the document.
- The following is part of the sample response:

```
<HTML>
<HEAD>
<TITLE>eXtropy Homepage</TITLE>
[...]
</HEAD>
</HTML>
```
- But this simple request/response protocol was quickly outgrown, and it wasn't long before HTTP was refined into a more complex protocol (currently version 1.1). Perhaps the greatest change in HTTP/1.1 is its support for persistent connections.
- In HTTP/1.0, a connection must be made to the Web server for each object the browser wishes to download. Many Web pages are very graphic intensive, which means that in addition to downloading the base HTML page (or frames), the browser must also retrieve a number of images. Many of them may actually be quite small and merely sliced up to provide some hard-coded formatting framework to the rest of the HTML page.
- Establishing a connection for each one is wasteful, as several network packets have to be exchanged between the Web browser and Web server before the image data can ever start transmitting. In contrast, opening a single TCP connection that transmits the HTML document and then each image one-by-one is more efficient, as the negotiation of starting new TCP connections is eliminated.

How Does a Web Server Serve Content?

- Of course, it's all very well to define HTTP as the protocol for a browser to communicate with a Web server. However, there is more to a Web server than its function as a communications protocol. **Ultimately, a Web server serves up content.**
- This content must be identified in a way such that a Web browser can download and display that content in correctly.
- The primary mechanism for deciding how to display content is the MIME type header.
- **Multipurpose Internet Mail Extension (MIME)** types tell a Web browser what sort of document is being sent. Such type identification is not limited to simple graphics or HTML.
- In fact, more than 370 MIME types are distributed with the Apache Web server by default in the mime.types configuration file. And even this list does not represent the entire universe of possible MIME types! MIME types are distinguished using a type/subtype syntax associated with a file extension. Here is a brief snippet from an Apache **/etc/mime.types** file.

```
text/xml      xml
video/mpeg    mpeg mpg mpe
video/quicktime qt  mov
```
- From this, we can see that files containing MPEG video content end with file extensions such as mpeg, mpg, or mpe. So a file with the name **"bhajan.mpeg"** would be served up as being an MPEG video file.

How Does a Web Server Accept Connections?

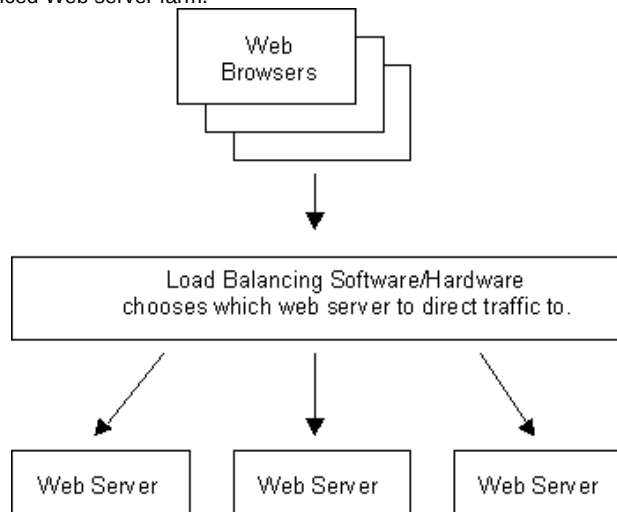
- Web servers are designed around a certain set of basic goals:
 - Accept network connections from browsers.
 - Retrieve content from disk.
 - Run local CGI programs.
 - Transmit data back to clients.
 - Be as fast as possible.
- Unfortunately, these goals are not totally compatible. For example, a simple Web server could follow the logic below:
 - Accept connection
 - Generate static or dynamic content and return to browser
 - Close connection
 - Accept connection
 - Back to the start...
- This would work just fine for the very simplest of Web sites, but the server would start to encounter problems as soon as clients started hitting the site in numbers, or if a dynamic page took a long time to generate.
- For example, if a CGI program took 30 seconds to generate content (certainly not an ideal situation anywhere, but not completely unheard of), during this time the Web server would be unable to serve any other pages.
- So although this model works, it would need to be redesigned to serve more users than just a few at a time. Web servers tend to take advantage of two different ways of handling this concurrency: multi-threading and multi-processing. Either they support the inetd module on Unix (which is a form of multi-processing), multi-threading, multi-processing, or a hybrid of multi-processing and multi-threading.

How Do You Choose a Web Server Platform?

- Early Web servers used `inetd` to spawn a Web server process that could handle each Web browser request. They were fairly simple applications, and there was no expectation of them having to cope with a high number of hits, so this was a totally reasonable design decision to make at the time.
- The easiest way to write a server application for Unix systems that needs to handle multiple connections is to take advantage of the `inetd` daemon, which handles all needed TCP/IP communication.
- Normally, a server process has to handle listening for and accepting TCP connections as they are made. It must then make the choice to either juggle concurrent connections, or effectively block any new connections until the current one has been fully served and closed. The `inetd` daemon can do all this instead, by listening on the desired port (80, by default, for HTTP requests), and running a Web server process as it received each connection.
- Using this method also makes administration of the machine easier. On most Unix machines, `inetd` is run by default, and is a very stable process. Web servers on the other hand, are more complex programs and can be prone to crashing or dying unexpectedly (although this has become less of a problem as these applications have matured). It also means that the administrator doesn't have to worry about starting and stopping the Web server; as long as `inetd` is running, it will be automatically run each time an HTTP request is received on the given port.
- On the downside, having a Web server process run for each HTTP request is expensive on the Web host, and is completely impractical for modern popular Web sites. These days, most Web sites run a Web server that supports either multi-processing or multi-threading, and are thus able to handle a much higher load.

How Do You Organize Web Servers for Performance?

- No matter how good your Web server is or how powerful a machine it is running on, there is always going to be a limit to the number of pages it can serve in a given time frame -- particularly if you are relying on a high percentage of dynamic content. Dynamic content typically relies on heavy database usage or processing of other program code, which takes up many server-side resources.
- Another problem comes up when running a Web site that has grown popular beyond its immediate means of serving content and ways to spread this load out -- usually across multiple machines, and sometimes across multiple sites.
- There are a number of ways to achieve **load balancing**. Perhaps the simplest way is to split the content across multiple hosts. For example, you could place all static HTML files on one host, all images on another, and have the third run all CGI scripts. Of course, this is a very crude form of load balancing and, depending on the content of the site, may have very little effect.
- For example, if a single CGI script is causing a bottleneck for a Web site, moving it to a separate server helps only the HTML, images, and other CGI scripts continue operating. Unfortunately, the heavily-loaded CGI script will still be a bottleneck for users who are utilizing that particular resource. Thus, load balancing on a Web server requires a bit more sophistication to figure out the right mix of where to migrate workload.
- In other words, a number of factors must be worked out before deciding on a correct solution for load balancing. In particular, examining the access patterns for the site is crucial to the performance tuning and load balancing process.
- The list below outlines some possible mechanisms used to spread the load among Web servers. Included also is a brief description of these mechanisms to provide a sense of how the Web server ties into external infrastructure and the clients that may direct traffic to them. The figure below illustrates the concept of a load-balanced Web server farm.



- DNS balancing (round-robin type)
- Hardware load balancing
- Software load balancing
- Reverse proxying
- Content spreading across hosts
- Content spreading across outsourced providers

Load Balancing - DNS

- DNS balancing is one of the easiest ways to create a Web site that can handle more hits. It basically involves having multiple copies of the site on separate physical servers. However, each server must be identical.
- Then, the DNS server for the hostname of the site such as `www.shiva.bom.labs.net` is set up to return multiple IP addresses for the site. The DNS server can do this by either just returning more than one IP address for the hostname or returning a different IP address for each DNS request it receives.
- Either way, what happens is a very basic distribution across the Web servers, although as far as the Web browsers are concerned there is only one Web site. This balancing is very basic, however, as it is difficult to determine to which IP address each client will resolve the site name. Also, since DNS query answers are essentially cached by the clients and other DNS servers, that single client will continue accessing the same Web server for the duration of the user's visit.

- It is possible then that through a luck of the draw, heavy Web site users may get one IP address, and less-frequent Web site users tend to get another IP address. Thus, even with this load-balancing technique in effect, it is possible that the Web server belonging to the first IP address will be highly loaded, and the other one will be lightly loaded, rather than having the load spread evenly between the two.
- Unfortunately the problems with this sort of "poor-man's load balancing" does not stop there. DNS Caches may not stay alive forever. So it is possible that a client, while using a Web site, may end up receiving a different IP address for the Web site. This can cause problems with dynamic sites, particularly ones that need to store data about the client.
- As it is possible for a single client to hit more than one of the Web servers, this data needs to be shared across all of them. Depending on how complex the data is, this may be a nontrivial programming task to get the data shared in real-time amongst all the Web servers equally.

Load Balancing -- Software/Hardware

- Software and hardware load balancing is similar to the DNS method just discussed, but rather than having the client attempting to access multiple IP addresses, only one is published. A machine is then set up to intercept HTTP requests to this one IP address and distribute them among the multiple servers hosting the Web site. Usually this distribution occurs at the level of TCP/IP routing which transparently maps a single source/destination IP address to a particular Web server in the Web farm.
- This can be done with both hardware or software, with hardware solutions generally being more robust, but of course more expensive. The balancing this achieves is usually much better than the DNS method, as the load balancer can distribute the requests more evenly across the Web servers.
- Also, these types of load balancers typically also occasionally detect when a Web server in the pool has gone down, and they can dynamically redirect the request to an identical Web server. With DNS load balancing, the client is stuck with a cached IP address of a downed Web server and cannot be redirected to a new one until the Web browser can request another IP address from the DNS server.

Load Balancing -- Reverse Proxying

- Another quick-win method of reducing load on a Web site is to use a reverse proxy, which intercepts requests from clients and then proxies those requests on to the Web server, caching the response itself as it sends it back to the client.
- This is useful because it means that for static content the proxy doesn't have to always contact the Web server, but can often serve the request from its own local cache. This in turn reduces the load on the Web server. This is especially the case when the Web server also serves dynamic content, since the Web server hardware can be less tuned to static content (when it is cached by a front-end proxy) and more tuned to serving dynamic content. It is also sometimes the case that although the Web server is serving dynamically created pages, these pages are cachable for a few seconds or maybe a few minutes. By using a reverse proxy, the serving of these pages speeds up dramatically.
- Reverse proxying in this manner can also be used alongside the simple load balancing method mentioned earlier, where static and dynamic content are split across separate servers. Obviously the proxy would be used on only the static content Web server.

Load Balancing -- Distributing Content

- Earlier, we talked briefly about distributing content among several Web servers and hard-coding their links. For example, if there is a bottleneck for bandwidth, the images on a Web site could be distributed across a couple servers (e.g., the documents for www.shiva.bom.labs.net could stay at www.shiva.bom.labs.net, while the images would be referenced as image1.shiva.bom.labs.net and image2.shiva.bom.labs.net if they were split among two separate servers).
- However, we also mentioned that this sort of "load balancing" is not really very dynamic and does not respond well to changing usage patterns that the previously discussed load balancing techniques deal with.
- One recent entry into the market of load balancing sites with heavy content to download (such as images) are service providers that specialize in hosting images, sound, multimedia, and other large files for distribution. These service providers have purchased disk space at ISPs worldwide and rent space on all of those ISPs.
- Then, these service providers use load balancing techniques that distribute the work of sending around the data files all over the world. Typically, these are more sophisticated load balancers that use a combination of DNS load balancing and software- hardware-based load balancing. They can tell where a user is geographically so that the images are served closest to the user. Thus, a user in Germany would get images stored on a German ISP, and a user in Hong Kong would get images served from a Hong Kong ISP.
- These server providers typically represent the most advanced in global load balancing technology, although as with the reverse proxying method mentioned above, the benefits seen from doing this will depend on how much static content you are hosting. Several such providers include **SandPiper** and **Akamai**.

How Do You Configure Web Servers for Performance?

- Of course, load balancing is really just a trick. We really have not improved the performance of a single Web server, we've just used external techniques to spread the load among many equivalent Web servers.
- The most obvious way to accelerate a Web server is to boost the resources available to it. These resources include disk speed, memory, and CPU power.
- Having reasonable CPU power is important for serving content; however, boosting it rarely helps a Web site unless the content being served is dynamic -- that is, the content is being generated by a program. Thus, most traditional content-based Web sites make use of the other techniques to increase Web server performance.
- Naturally, the first step to serving content, such as an image or an HTML document, is to retrieve it from disk. Thus, it makes sense that disk access speed is increased. However, disk configuration helps only so much. Ultimately, physics gets in the way, and you cannot retrieve data off of a rotating platter any faster. Thus, most Web sites get the best benefit out of adding raw memory to the server. The more memory a Web server has, the longer Web pages and images can be cached in memory instead of read from the disk over and over again.
- Another technique is used to cache images and data is **proxying**. We talked about **proxies earlier within the context of load balancing** in front of several Web servers.
- **In the context of improving performance** on a single Web server, an accelerating Web proxy sits in front of a traditional Web server and acts as a pipe that sucks down the content from the Web server and keeps it in memory. High performance proxies also go so far as to store the images and data they cannot keep in memory in a highly indexed data store on disk.
- The difference between this data store and the Web site itself is that the data store is structured for extremely fast retrieval of content. A Web site directory structure on the other hand, is optimized for the organization of data from a human point of view.
- Finally, e-commerce servers have another bottleneck to contend with: **encrypting SSL transactions**. It turns out that establishing SSL connections is very expensive in terms of CPU power. This is another case where having a fast CPU will help you. However, there are better solutions, like **SSL acceleration**.

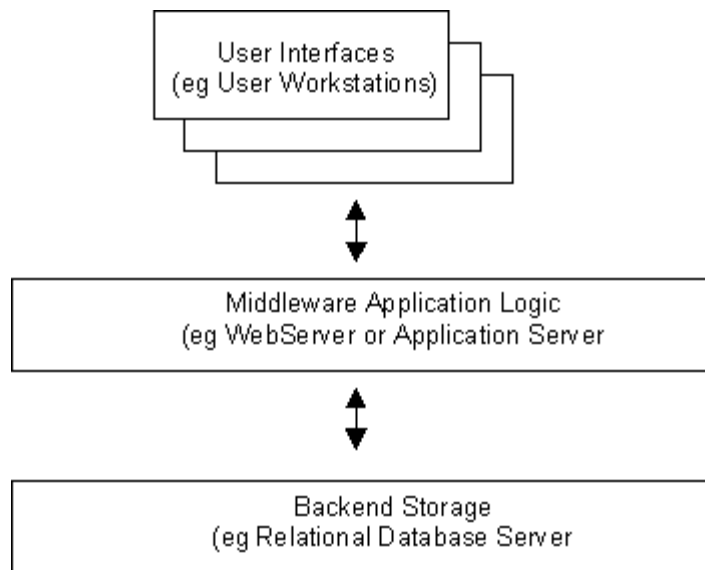
- Rather than spending money on the highest Mhz processor, enterprises can buy from one of the third-party vendors that sell **SSL acceleration cards** and **add-ons** to Web servers. These are processors that are inexpensive yet highly optimized to perform encryption extremely rapidly.
- Generally these cards offer another advantage in that the SSL keys for the Web server can usually be stored in the card. **A dip switch can then be flipped on the card to make that SSL key irretrievable.** This is important because if an SSL Web site is hacked into, it is possible for an intruder to steal the SSL keys. With physically locked SSL encryption hardware, there is no way an intruder can get to those keys.

How Does Web Server Security Work?

- There are two different levels of security for a Web server.
- On one level is the security of the data stream itself so that it may not be viewed or modified by a malicious third party.
- On another level is the security of the content itself -- the authentication and authorization of people to view and change that content.
- As we mentioned earlier, URLs that begin with "https" are handled using SSL (now referred to as Transport Level Security -- TLS) algorithms. These algorithms basically work by setting up a secure, encrypted link between a Web browser and a Web server.
- However, you might ask, what is SSL protecting anyway?
- There are really only two choices:
 - SSL is protecting either the data being posted to the Web server or
 - the retrieval of some confidential data from the Web server.
- An example of a user posting confidential data to a Web server can be found in a typical Web store application. In such an application, the user is usually given a choice of presenting his or her credit card information to the order form.
- Although the Web server may not echo the user's credit card information again to the Web browser, the actual transmission of this information must to be treated as confidential.
- Then, there is the issue of protecting content on the Web server that is being served. For example, an auction site may want to protect the bids a user is receiving for an item so that only the individual who posted the item sees all the bids.
- In this case, it is not enough to simply encrypt the traffic being sent. The Web server must also be able to identify the user and the data she has access to. These two processes are referred to as **authentication** and **authorization**, respectively.
- Web servers usually support authentication using a technique called **Basic authorization**. In this technique, a Web server sends a special header to the user's Web browser asking for a username/password combination. This results in the Web browser popping up a log-in window. More on this later, in great detail.
- Web servers are usually less sophisticated with regard to authorizing the viewing of data. Most Web servers merely allow the restriction of the reading of directories and files within a directory by group or user. More sophisticated options for determining whether a user is authorized to view files (such as time of day) must usually be programmed into a Web application.

How Does a Web Server Differ From an Application Server?

- We've already talked about what a Web server can do. But what about an application server? The distinction used to be quite clear. A Web server only served up HTML and images for viewing on a browser. And while an application could exist on the Web server, that application was typically restricted to just generating HTML and image data.
- Likewise, in the beginning, the definition of an application server was fairly concrete. An application server merely contained raw business/application logic of an application and did not contain database or user interface code.
- In many cases, the application server served as the middle-tier of three-tier programming. The figure below contains an illustration of the three-tier programming model.
- In other words, an application server sits in the middle of other programs and serves to process data for those other programs. Usually, in the case of three-tier programming, the two layers that are separated by the application server is the User Interface layer and the Database/Data Storage layer.



- Note that the concept of an application server should not be confused with Web applications sitting on a Web server itself. Web applications generally also contain application logic, but since they primarily serve to generate HTML for a Web browser, they are also user interface related and generally do not ever reside on a pure application server.

Data Marshalling

- Data marshalling is a term used to refer to the way applications talk to each other. **Similar to how Web servers wrap human-readable content in HTML to make it palatable to the eye, application servers wrap application-readable content inside other tags to allow the data to be interpreted by the receiving application.** These "tags" formed the standards around which application servers were formed. For example, **CORBA** servers use a protocol called **IOP (Internet Inter-Obj Protocol)** to transfer data between application objects.
- More recently, **XML (Extensible Markup Language)** has made data marshalling as easy as making up your own tags that are similar in syntax to HTML except that they describe data rather than how content should be displayed in a browser.
- In the past, most such application servers talked the language of data marshalling protocols such as **IOP for CORBA, Java's object serialization for RMI, and DCOM for remotely activating Microsoft ActiveX objects.** However, the rise of XML (extensible markup language) as an internet friendly data marshalling language has blurred the boundaries.
- Web Servers are turning into application servers that serve XML data alongside HTML data. Likewise, application servers are being marketed as being able to add value by having the capability of acting as a simple Web server while still delivering on its core application server functionality.
- Nearly all **Java Enterprise Bean servers** market the capability to simultaneously serve **Java Servlets** and **Java Server Pages** -- traditionally the realm of Web servers. Likewise, new data marshalling languages such as the **Microsoft endorsed SOAP (Simple Object Access Protocol)** XML standard have implementations that run in conjunction with existing Web servers.
- **So what should you use now that the distinction has become blurred? Web Server or Application Server?**
- The reality is that one size does not fit all. **Typically, application servers are tuned for processing data using objects or code that represents application logic. Likewise, Web servers tend to be tuned to sending out data.**
- Web sites rule of thumb is that if you think of your Web site as, well, a Web site, then you should probably be using a Web server. Even if the data you are serving is dynamic, the fact that your site is Web centric means that all the configuration parameters of your Web server are best served tuning how you display information to other people's browsers.
- For example, although it is easy to find an application server that can serve Web pages, you will be hard-pressed to provide one that supports Server-Side Includes (SSI) which is a feature nearly every Web server supports out of the box. Of course, you can still add application server components to a Web server if part of the data will be related to applications.
- However, if you find that you are using application server components as the primary reason for your Web site, or that the Web server itself is being dragged down by all the resources that the application components may be using, then you should consider moving the application components to their own application server.
- One thing we should mention is that breaking out application components of a Web site into a separate application server can help you in several ways. In some cases, breaking out the application components can increase both performance and stability of an application. However, you should be wary of doing this too early because the addition of yet another server in your environment can also add quite a bit of complexity to managing your infrastructure.
- As just mentioned, breaking out application components in this way also usually aids in performance. For example, Web servers are usually highly tuned to serve data from disk such as HTML pages and images very efficiently. **Such a server is tuned to speed up IO operations.** On the other hand, application objects are usually operating on pure logic alone -- they take data in from a stream, process it, and send new data back out again. This is a CPU rather than IO intensive activity. **Thus, the application server is best served when it is tuned for CPU usage.**
- In addition, breaking out components usually adds to the stability of an application. Application servers are tested by their respective vendors to work in the context of executing application logic and so they are thoroughly debugged in that context. Likewise, Web servers are heavily tested within the context of serving documents either dynamically or statically. Mixing the two into one server can cause unexpected bugs that neither vendor has tested for.
- Note that the word vendor used loosely here since Apache has no "vendor" but rather is a community of open source developers who have arguably tested Apache beyond the scope that many other Web server vendors test their own products. However, even an Open Source community may have limits to the mix of environments that their products have been tested in.
- Of course, these benefits should be tempered by what was mentioned earlier. If your Web site is mostly serving document data, it probably doesn't make much sense to rush out and purchase an application server. Indeed, if your environment is simple, you could be adding unnecessary complexity to the overall solution, thus making it harder to maintain and administrate -- unless you have devoted human resources to such an endeavor. As a rule of thumb, the more servers there, the more that must be maintained.

How Does a Web Server Run Web Applications?

- A discussion of application servers and Web servers would not be complete without a brief introduction to some of the technologies used to allow a Web server to serve dynamic data. In a way, these technologies are what led people to realize that Web servers can also serve application data such as XML instead of merely serving up HTML documents.
- Although these technologies may appear to be close in definition to an application server, they are different. Web application technologies are primarily focused on delivering dynamically generated HTML documents to a user's Web browser while they interact with a Web site. The pure application servers do not format data for humans to read. Rather they act as an engine that processes data for another program to read and interpret on behalf of the user.
- Let us examine some of the following technologies:
 - CGI
 - Microsoft ASP
 - Java Servlets
 - PHP
 - mod_perl
- The first such technology is **Common Gateway Interface**, commonly referred to as CGI. As previously discussed in the section that focused on what a Web server does, CGI programs serve HTML data dynamically based on input passed to them. Many people associate CGI with a language called Perl because Perl has been used to create a majority of the CGI scripts that currently in use.
- However, CGI is not language specific -- it is merely a protocol for allowing the Web server to communicate with a program. CGI can be written in any language, and common choices, in addition to Perl, include C, Python, and TCL.
- The disadvantage of CGI is that it tends to be slow because each request for a dynamic document relies on a new program being launched. Starting new processes on a Web server adds extra overhead. Soon after CGI came into being, other technologies quickly followed to solve this performance issue.
- **Microsoft Active Server Pages (ASP)** technology consists of embedding a **VBScript** interpreter into the **Microsoft Internet Information Server**.

- On the **Java front, Servlets and Java Server Pages** connects a perpetually running **Java Virtual Machine** to a Web Server. Servlets have an additional advantage over ASPs in that they become cached in the Java Virtual Machine after their first execution. VBScript pages are reinterpreted each time they are hit.
- In the Open Source community, **PHP** (<http://www.php.net/>). It is similar to JSP and ASP technology in that PHP consists of a set of additional code tags placed inside of existing HTML documents. The interesting part about PHP is that it is a language developed purely to serve Web pages rather than being based off of an existing language such as Perl, Python, Visual Basic, or Java. This makes PHP-written applications very succinct compared to equivalent VBScript or JSP applications.
- While all of this was going on, the Perl community did not rest on its laurels. All of today's major Web servers have Perl acceleration solutions available to it. Apache has a free solution called **mod_perl**, which embeds the Perl interpreter inside of Apache. Not only does this speed up Perl scripts, but the scripts themselves are also cached by mod_perl, providing further performance boosts.
- mod_perl also hooks tightly into Apache so a Perl developer can literally change the behavior of how the Apache Web Server works. Previous to mod_perl, this type of control over a Web server belonged solely to the realm of C programmers coding to the low-level Apache API.
- Commercial solutions from **ActiveState** and **Binary Evolution** also exist to accelerate Perl in a similar way to mod_perl. ActiveState's product, **PerlEx**, accelerates CGI/Perl scripts for Internet Information Server while Binary Evolution's product exists for Netscape, Apache, and Internet Information Server on both Unix and NT platforms.
- Phew! With all these fast solutions around, it may seem like it's a bit hard to choose a language to in which to develop when making Web servers serve up dynamic pages. Actually, it's quite easy. Unless you have a really large hit rate where squeezing every performance bit out helps, you should go with whatever language is comfortable for you or your organization to do its job effectively.

Wrapping Up

- That's it! How a Web server works in a nutshell. Well, OK, so it's a pretty big nutshell. On the surface, it is difficult to believe that there can be so much to a piece of software that distributes HTML documents and images.
- Seven years ago that was the case. Since then, the concept of content has evolved to include application data, streaming multimedia, security models, and integration with other data and content servers. Likewise, the mechanisms behind how the Web server integrates with the rest of the world to provide this content has expanded as well.
- In summary, this tutorial has gone over all the bits and pieces that make a Web server work. We have covered everything from how the server works internally to how it works within an existing infrastructure.

Want Next?

- Now we dive into the technical details of how to Configure a Web Server using Apache.
- So, fasten up!

Setting Up a Web Server

- The configuration directives are grouped into three basic sections:
- 1. **Global** : Directives that control the operation of the Apache server process as a whole (the 'global environment').
- 2. **Main** : Directives that define the parameters of the 'main' or 'default' server, which responds to requests that aren't handled by a virtual host. These directives also provide default values for the settings of all virtual hosts.
- 3. **Settings for virtual hosts**, which allow Web requests to be sent to different IP addresses [ip-based] or hostnames [name-based] and have them handled by the same Apache server process.

Section I → Global Environment

- ServerType Standalone or xinetd : Deprecated.** ServerType defines how the server is started. If the server starts from a startup script at boot time [like ours - System V-style], the option is **standalone**. If the server runs, on demand, by the xinetd daemon, i.e. via a TCP-wrapper, then it is **inetd**. Most of the time Web servers are very busy, so standalone is a good option since the service is started at boot time once and for all. If you have a rarely used site then inetd is OK. As of Apache 2.0, **standalone** is the only choice.
ServerType standalone
- ServerRoot "/etc/httpd"** Do NOT add a slash at the end of the directory path. All references to files henceforth will be from here and below viz., log files, config files etc Configuration and logfile names: If the filenames you specify for many of the server's control files begin with "/" (or "drive:/" for Win32), the server will use that explicit path. If the filenames do *not* begin with "/", the value of ServerRoot is prepended -- so "logs/foo.log" with ServerRoot set to "/etc/httpd" will be interpreted by the server as **"/etc/httpd/logs/foo.log"**.
- LockFile** : This is used only if the "logs" dir is NFS mounted, since the lockfile must be stored on a local disk, which contains the PID of the parent httpd process. This PID is auto appended to the filename. You may want "logs" dir to be NFS shared bcos many people can then run statistics on the multiple vhosts to enable client reporting and thus easing reporting load.
LockFile /var/lock/httpd.lock
- PidFile**: Where parent daemon PID is stored. Used only in standalone mode. Used when you wish to send a the server a signal so it closes and then reopens its ErrorLog/TransferLog and re-reads its config file. Send a HUP (kill -I) signal to PID given in this file.
PidFile /var/run/httpd.pid
- ScoreBoardFile**: Used by server parent daemon to communicate with its children and other runtime process info management information.
ScoreBoardFile /var/run/httpd.scoreboard
- ResourceConfig /dev/null** **conf/srm.conf }** Both are deprecated
- AccessConfig /dev/null** **conf/access.conf }**

Managing Child Process And Performance-Tuning

8. Timeout 300	12. MinSpareServers 5	16. MaxRequestsPerChild 1000
9. KeepAlive Off	13. MaxSpareServers 20	17. ThreadsPerChild
10. MaxKeepAliveRequests 100	14. StartServers 8	18. ListenBackLog 511
11. KeepAliveTimeout 15	15. MaxClients 150	

- StartServers** : Since no.of child processes is dynamically controlled depending on load, no point in adjusting this parameter. It is the no of children daemons started by the parent Apache processes. The server process, so as to speak. Each of these children run as **"apache/apache"** and handle client requests when you ask for a WebPage (request/hit) . A child is allocated to you. It is then killed by the parent , a newchild (PID) is spawned which then will handle your next click on the WebPage(hit/request). So on every click on the WebPage, a new child is allocated to handle your request. Good for security, since a new PID(child) is generated everytime. That's why you get a "Connecting to..... " msg, but decreases performance. Why not have a single child be kept open, servicing all your requests(hits). This is called a **persistent** connection and enabled by **9. KeepAlive On**
 - KeepAlive, an extension to HTTP, provides a persistent connection between browser and server so that the same connection can handle multiple request/response pairs. The result is a drop in latency, or the time taken up by establishing a connection.
 - Now one child will services all your hits, but this could be bad. So a **10. MaxKeepAliveRequests (100)** will be allowed per persistent TCPconnection. The child will then be killed and another will be spawned to handle your further requests. You will get a "Connecting" msg again.
 - NOTE** : If 0 , unlimited requests will be allowed. Keep a high value for max server performance.
 - So you are allowed a 100 clicks on a Website on the same child daemon. But what if you take a long time to complete those 100 hits? Another crackerpot! So if you hesitate more than **11. KeepAliveTimeout (15)** secs between successive clicks once again a new child will be reallocated killing the older one.
 - NOTE**: Too high a value causes problems in heavily loaded servers. The higher the timeout, the more processes will be kept occupied waiting on connections with idle clients.
 - Now suppose you do click links within 15 secs and have clicked about 30-40 of them, so far, you have a persisten connection or the same child daemon is with you from the start. Now if you make a hit and page take more than **8. Timeout (300)** secs to load once again the current child will be killed and a new one will be spawned. Keep reasonably low but for slow modems keep it reasonably high.

- Basically, the number of seconds the server waits for a transfer to complete. The value needs to be high enough to handle the size of the files your site sends and the low performance of the modem connections of your clients. But if it is set too high, the server will hold open connections for clients that may have gone offline.

Timeout waits for:

1. Total amt of time it takes to receive a GET request
2. Amt of time between receipt of TCP pkts on a POST or PUT request.
3. Amt of time between ACKS on transmissions of TCP pkts in responses.

- Soon these will be 3 separate directives in a future release
- IMP NOTE:** The above process will occur only if **KeepAlive** is **On**

12. MinSpareServers :

- NOTE :** Setting this to some "m" ensures that you will always have atleast "n+m" httpd processes running when you have "n" active client requests. i.e. Since each client has a child processes + these idle process.

- Min no. of idle processes to maintain at all times. An idle process is the one that is not handling any requests. If there are fewer than **MinSpareServers** idle, parent creates new ones at a rate of 1 per sec to maintain the specified value.

MinSpareServers **5**

13. MaxSpareServers :

- Max no of idle processes to maintain or max no of spare servers. If there are more than **MaxSpareServers** idle, then they are killed off by the parent process. Increase only on very busy sites.

MaxSpareServers **20**

- So far we have seen that each persistent connection handles one client who may/may not have over 100 requests per persistent connection.
- Hence the total number of children running [child processes] at any given time on the server will be equal to the number of "persistent" connections or the no of clients connected to your WebServer (connections).

15. MaxClients (150) sets the limit on the max total no of simultaneous or persistent connections (read connections or Clients connected) that can be supported. Requests beyond this are queued until a server is available. Max is 256 or else recompiled.

- Since each persistent connection (or non-persistent too) is handled by ONE child daemon, not more than this no. of child server processes will be created. Hence, this is also the max no of children alive at the same time. This number cannot be exceeded. Clients are locked out. Brake to keep a runaway server from taking system with it down.
- Any connections attempts over MaxClients limit will normally be queued up to the no specified by **18. ListenBacklog 511**. Once a child process is freed at the end of a different request, the connection will then be serviced.
- In short, hence, the max length of the queue pending connections is specified by this directive. No tuning is reqd, but can be increased when under a TCP SYN flood attack.

16. MaxRequestsPerChild : default : 0 Unlimited. Process will never die (eg 1000)

- We know that for persistent (KeepAlive) connections, the max no of requests allowed is **MaxKeepAliveRequests**. When this directive is specified it sets the limit on the number of requests(hits/clicks) that an individual child will be handle before it dies.
- This directive seems like a redundant directive since "**MaxKeepAliveRequests**" seems to do the something BUT, this alters the behaviour of the no of persistent requests a connection (child) can handle. Now, here in this case for persistent connections only the first request is counted towards this limit. Hence, effectively this directive allows for a huge increase in the no. of requests that a child process can handle. Setting this to non-zero is beneficial bcos:
 - It limits the amnt of memory that the process can consume by accidental memory leakage.
 - By giving processes a finite lifetime helps reduce the no. of processes when the server load reduces.
- On a Win32PC, you should use **17. ThreadsPerChild" (0)**. Win32 handles additional connection requests with threads within one child process rather than fork additional process.

19. Listen 80 : By default Apache listens in on all NW interfaces (NIC) connected on your server but only on one Port - the one specified by the **Port** directive. This directive allows you make Apache listen on more than one IP address(NIC) and/or Port or a combination.

- Eg **Listen 80** **Makes server accept connections on both port 80 and 8000**
Listen 8000
- Eg **Listen 192.170.2.1:80** **Makes server accept connections on two specified interfaces and port nos**
Listen 192.170.2.5:8000

In the above case the **Port** and **BindAddress** directives will be ignored totally. **Both have been deprecated anyway in Apache 2.0**

Hence if you specify **Listen** with:

- An IP address but no port, **Port** will be used.
- A Port but no IP Address, **BindAddress** will be used.
- No "Listen" directive given at all, then Apache will listen in on **BindAddress**.
- If no **BindAddress** or **Port** directives are given also, then Apache will listen to ' * ', all NIC interfaces on Port 80, by default.

- 20. BindAddress : Deprecated.** By default Apache listens on all IP addresses. With this directive, you can curtail this and have Apache listen only on this address. Use the more flexibe **Listen** instead. Deprecated in Apache 2.0

21. **LoadModule:** (eg) `LoadModule foo_module modules/mod_foo.so` ---> server root `/etc/httpd/modules/mod_foo.so`

Syntax: `LoadModules module filename`

- Loads the module "module" contained in "filename". The order in which modules are loaded is v. imp.
22. `<IfDefine>`
 some module
`</IfDefine>`
23. **ClearModuleList** **Deprecated**
24. **AddModule** **Deprecated**
25. **ExtendedStatus** **Def: Off**
- This directive controls whether the server keeps track of extended status information for each request. This is only useful if the status module is enabled on the server. This setting applies to the entire server, and cannot be enabled or disabled on a virtualhost-by-virtualhost basis. You can access server statistics by using a Web browser to access the page <http://your.server.name/server-status>
26. **Include conf.d/ *.conf**

Example 1

- Try this in X in Konqueror : <http://localhost/usage> or <http://shiva.bom.labs.net/usage>
- This uses Webalizer to analyze traffic stored in `/var/www/html/usage`

Section II → 'Main' server configuration

27. **Port :** **Deprecated** In the absence of any Listen or BindAddress directed, Apache listens in on this Port value which defaults to 80, a privileged port. The **ServerName** and **Port** together specify the canonical address of the server. [socket]
28. **User apache**
29. **Group apache**
- The children started by the parent Apache daemon which service the actual Web requests, run with a "uid/gid (48/48)" and "apache/apache" user/group.
 - **NOTE :** Never run the server processed as 'root' although you will have to be "root" to start it. Even if a cracker gets in, s/he will not have a login shell and max have user **apache** privileges, which should ideally be none on the system.
30. **ServerAdmin :** Address of Web Server Admin, where problems with the server should be e-mailed. Mostly set to root. Change to whoever is Web Admin. This address appears on some server-generated pages, such as error documents.
ServerAdmin webmaster@shiva.bom.labs.net
31. **ServerName :** Most Important Directive. Sets host name of server. Can be different from canonical (actual) name of the m/c. If not given would deduce it to be www.example.com (**hostname.domainname**) if the canonical name of the m/c were "something.example.com"
ServerName shiva.bom.labs.net
32. **DocumentRoot :** Where httpd will serve files from. Server appends path from requested URL to doc root to make path to documents.
DocumentRoot `"/var/www/html"`

Blocking Access to Sites : [who can access files in the doc root of VHs]

There are 3 ways to block access to a Site :

1. Scoping and Scopes
2. HTTP Authentication
 - a. With **htpasswd**
 - b. With **dbmmanage**
 - c. With **htdmb**
3. Order directive

SCOPING AND SCOPES

- 6 directives control the scope of the what can and cannot be accessed. They are :
 32. `<Directory> ..... </Directory>`
 33. `<DirectoryMatch>..... </Directory>`
 34. `<Files> ..... </Files>`
 35. `<FilesMatch> ..... </Directory>`
 36. `<Location> ..... </Location>`
 37. `<LocationMatch> ..... </Location>`
33. `<Directory> ..... </Directory>`
 A scope in the Apache config file can contain any number of directives which will affect only the directory scope it specifies. Any number of scopes can be specified.
<Directory />
 Order Deny,Allow
 Deny from all

</Directory>

- The / dir and all dirs inherited below it will inherit all the perms. The **Order** directive instructs httpd to consider **Deny** directives in this scope before **Allow** directives. The **Deny** directive here blocks access to all files in this scope.
- This effectively disables any access to any part of the Web server. Clearly this is not desirable bcas a Web server is pointless if it is not allowed to actually server any pages.
- So how do Users access their pages?
- By making exceptions and selectively allowing access on a per-directory basis and overriding.

Example 2

```
<Directory /web/www.bollywood.com>
    Order Deny,Allow
    Allow from all                Grant public access
</Directory>
<Directory /home /*/public_html>
    Order Deny,Allow
    Allow from all
</Directory>
```

34. **<DirectoryMatch>** **</Directory>**
Use this if more flexibility is desired, since it allows the use of REs in the arg.
35. **<Files>** **</Files>**
Analogous to <Directory> section, but is used to protect a **single file**, which can be supplied relatively or with an absolute path.

Example 3

To protect file **private.txt** from being accessed from anywhere in the system, except that **foo** can do it.

```
<Files private.txt>
    Order Deny,Allow
    Deny from all
    require user foo    Remove this line for total non-access of this file
</Files>
```

Example 4

To allow any file that ends in **.public.html** to be displayed regardless of where it is on the system

```
<Files *.public.html>
    Order Allow,Deny
    Allow from all
</Files>
```

Example 5

```
<Files Jan??.htm>
    order deny, allow
    deny from all
    allow from example.com
</Files>
```

- Only clients from **example.com** NW will be able to access these files and that too files of type **Jan??.htm*** . i.e directives will apply to these matched files. REs are allowed in matching using the '~' character

Example 6

```
<Files ~ "[jJ] Jan.*\..htm.*">    (files like Jan.htm, January.html, Jan1999.htm etc)
    order deny, allow
    deny from all
    allow from example.com
</Files>
```

36. **<FilesMatch>****<Files>**
- Consider the example below. This supercedes the above in that no '~' is needed now. Much easier.

Example 7

```
<FilesMatch "[jJ] Jan.*\..htm.*">
    order deny, allow
    deny from all
    allow from example.com
</FilesMatch>
```

37. **<Location>****</Location>**
- <Location> and </Location> tags allow you to specify access control based on URL.
 - If you want to allow people connecting from your domain to see server status reports, you should uncomment the next section of directives:

Example 8

```
#<Location /server-status>
#    SetHandler server-status
#    Order deny,allow
#    Deny from all
#    Allow from .bom.labs.net
#</Location>
```

- Now, in Example 1, we allowed everyone to see the status of the the server with the **usage** URL. We could enable/disable this with the

Location tag.

38. **<LocationMatch></LocationMatch>**
- Use this if more flexibility is desired, since it allows the use of REs in the URL concerned
39. **Options** - Enabling server Options
- These are normally given in the **<Directory></Directory>** scoping.
 - These handle Apache's security **aspect**. Let us examine these in depth!

40. None	44 Includes	48 SymLinksIfOwnerMatch
41 All	45 IncludesNOEXEC	
42 FollowSymLinks	46 Indexes	
43 ExecCGI	47 MultiViews	

40. **None** : Don't allow any of these options in the specified directory.
41. **All : Default. Enables** all options except Multiviews which has to be explicitly done with – **Options All MultiViews**
42. **FollowSymLinks** : Treats a symlink as if it were a document in the directory and it potentially allows the Web Client to escape from the Web doc dir and access a doc elsewhere in the file system.
- **Note**: Even though the server follows the symlink it does *not* change the pathname used to match against <Directory> sections.
 - **Note**: This option gets ignored if set inside a **<Location>** section.

SERVER SECURITY

- There are 2 ways to deny and allow access to users :
 - **An HTTP authentication scheme**
 - **IP-based or hostname-based access control**
 - Let us examine the first type firstly.
 - CGI and SSI are the greatest threats to an Apache Web Server. Be wary about enabling them.
 - Use these server-side options with care. All of them except MultiViews cause potential security problems, and all of them use server resources.
 - Let us discuss this. For **ServerSide/CGI Programmer's Only**.
43. **ExecCGI** : This cmd enables execution of CGI scripts within directory scope specified i.e CGI scripts in this directory may be executed. To make it easier to review all CGI scripts, keep them all in the **ScriptAlias** directory which is used specifically for executable CGI scripts. Do not allow **ExecCGI** in any other directory unless you have reviewed all the scripts that are supposed to be there.
44. **Includes** : Enable SSI or Parsed HTML [.shtml] processing by **mod-include** within the directory scope. Such files include other files or execute code from script files. If user input is used to dynamically modify the SSI file, it is vulnerable to the same type of attack as CGI scripts. SSI cmds are embedded inside HTML comments and begin with a **<!--** and ends with a **-->**
- SSI cmds are :
- #config** - Formats the display of file size and time
 - #echo** - Display variables
 - #exec** - Executes a CGI script or a shell cmd
 - #lastmod** - Display the date a doc was last modified
 - #size** - Display the size of a doc
 - #include** - Inserts another file into the current document.
45. **IncludeNOEXEC** : The safest way to operate a server is to disable SSI processing which is the default unless **All** or **Includes** is specified by an **Options**. A compromise setting is to allow SSI but to disallow the **"#exec"** **"#include"** cmds, which are the greatest threats. This command does just that.
- **Note** : The above 3 commands below are dangerous: Either disable with **"Options None"** or **"Options -Includes -IncludesNOEXEC -execCGI"**. This will selectively disable SSI processing for the scope without affecting any other existing options that may have been enabled.
46. **Indexes** : If a dir is requested and there's no file in that dir matching the DirectoryIndex directive, a listing of available files will present to the client in a formatted gopher style. That is, if a URL which maps to a directory is requested, and there is no **DirectoryIndex** (e.g., **index.html**) in that directory, then the server will return a formatted listing of the directory. [aka gopher-style]
47. **MultiViews** : Enables content negotiation for the scope, permitting the server to infer the correct or best doc name from a list of possible choices viz language to use when 2 or more have been selected.
- Too complex to explain here!** See the manual in /var/www/manual/ for content-negotiation.html and MultiViews
48. **SymLinksIfOwnerMatch** : Allow symlinks to be followed only if the target of the link is owned by the same user as the dir itself. In plain words, the symlink is followed only if the owner of the link is the same as the owner of the doc to which the link points. The server will only follow symbolic links for which the target file or directory is owned by the same user id as the link.
- **Note**: this option gets ignored if set inside a **<Location>** section.

EXAMPLE 9

Consider the following example:

```
<Directory /www/htdocs/foobar>
  options FollowSymLinks
</Directory>
```

```
<Directory /www/htdocs/foobar/subdir>
  options +ExecCGI
</Directory>
```

- One option is defined in the first dir scope. The propagates by default to all subdirs under it.
- But we wish to have ExecCGI for the lower level subdir
- If there was no '+', then all above values inherited would be overridden(null) and only this one -ExecCGI would apply.
- The '+' implies that ExecCGI will be appended to already existing options. Hence its importance.

EXAMPLE 10

Consider the following example:

```
<Directory /www/htdocs/foobar>
  options FollowSymLinks
</Directory>
<Directory /www/htdocs/foobar/subdir>
  options +ExecCGI
</Directory>
```

- One option is defined in the first dir scope. The propagates by default to all subdirs under it.
- But we wish to have ExecCGI for the lower level subdir
- If there was no '+', then all above values inherited would be overridden(null) and only this one -ExecCGI would apply.
- The '+' implies that ExecCGI will be appended to already existing options. Hence the importance of +

EXAMPLE 11

For example, without any + and - symbols:

```
<Directory /web/docs>
  Options Indexes FollowSymLinks
</Directory>
<Directory /web/docs/spec>
  Options Includes
</Directory>
```

Then only **Includes** will be set for the **/web/docs/spec** directory. All else will be nullified. However, modifying the above, using the **Options** directive + and - symbols:

```
<Directory /web/docs/spec>
  Options +Includes -Indexes
</Directory>
```

then the options **FollowSymLinks** is carried fwd and the **Includes** cmd is also appended for the **/web/docs/spec** directory. **Note:** Using **-IncludesNOEXEC** or **-Includes** disables server-side includes completely regardless of the previous setting. The default in the absence of any other settings is **ALL**.

Restricting Access to sensitive contents Using HTTP Authntication

49. AllowOverride

- When the server finds an **.htaccess** file (as specified by **AccessFileName**) it needs to know which directives declared in that file can override earlier access information.
- The **AllowOverride** directive sets whether or not any **Options** can be overridden by the declarations in an **.htaccess** file.
- By default, both the root directory and the DocumentRoot are set to allow no **.htaccess** overrides.
- **Override** can be set to **None**, in which case the server will not read the file, **All** in which case the server will allow all the directives, or one or more of the following given below. [These are also normally given in the **<Directory></Directory>** scoping as also the **.htaccess** file.
- What activities may be overridden ie what Overrides are allowed, is controlled by the **AllowOverride** directive, which has the following keywords:
 - 50 All
 - 51 None
 - 52 AuthConfig [These all below given in the .htaccess file]
 53. AuthName
 54. AuthType
 55. AuthUserFile
 56. AuthGroupFile
 57. AuthDBUserFile
 58. AuthDBGroupFile
 59. AuthDBMUserFile
 60. AuthDBMGroupFile
 61. require
 62. Limit
 63. LimitExcept
 64. FileInfo
 65. Indexes
 66. Options
- Hence, **AllowOverride** specifies what part of the config can be overridden in **.htaccess** files i.e directives in **.htaccess** files are executed only if their use is permitted by whats called the current override setting. So called bcas the enabled directives can be used to override a particular kind of activity.

- 50. **All** : All **settings** in **.htaccess** are processed or honored

51. **None : Completely** disables processing of .htaccess files in the scope. Totally ignores it.
52. **AuthConfig** : Indicates that the foll directives having discretionary access controls will be processed if found in the .htaccess files or "in the directory scope" given by <Directory></Directory>.
53. **AuthName** :
- This directive sets the name of the authorization realm for a directory. This realm is given to the client so that the user knows which username and password to send. **AuthName** takes a single argument; if the realm name contains spaces, it must be enclosed in quotation marks. It must be accompanied by **AuthType** and **require** directives, and optionally directives such as **AuthUserFile** and **AuthGroupFile** to work.
AuthName "Hitlers hit list"
54. **AuthType** :
- This directive selects the type of user authentication for a directory. Only **Basic** and **Digest** are currently implemented. It must be accompanied by **AuthName** and **require** directives, and optionally directives such as **AuthUserFile** and **AuthGroupFile** to work.
AuthType Basic
55. **AuthUserFile** :
- Location of username/ passwd file. Mostly this file is **.htpasswd** but could be anything.
 - This is given in the **.htaccess** file or by the file specified by the **AccessFileName** directive. [Def] **.htaccess**
Eg. **AuthUserFile /usr/local/.htpasswd** This file is for this VH dir scope only.!!! IMP
 - The **AuthUserFile** directive sets the name of a textual file containing the list of users and passwords for user authentication. Filename is the path to the user file. If it is not absolute (i.e., if it doesn't begin with a slash), it is treated as relative to the ServerRoot ie **/etc/httpd/**
 - Each line of the user file file contains a username followed by a colon, followed by the crypt() encrypted password.
 - Use the **htpasswd** command to maintain this password file.
htpasswd -c /usr/local/.htpasswd foo
 - Creates a password file 'Filename' with 'username' as the initial ID. It will prompt for the password.
htpasswd /usr/local/.htpasswd foo
 - Adds or modifies in password file 'Filename' the 'username'.
 - Note that searching large text files is very inefficient; **AuthDBMUserFile** should be used instead. We shall see that soon.
 - Security:** make sure that the **AuthUserFile** is stored outside the document tree of the web-server; do not put it in the directory that it protects. Otherwise, clients will be able to download the **AuthUserFile**.
56. **AuthGroupFile** :
- Location of file containing listings of user grps and members of those groups. If no such file is used, then specify foll so that no grp file is used.
AuthGroupFile /dev/null
- Creating user grps allows you to specify a larger no of people who are permitted to view a resource, but allows each to have his own name/passwd (.htgroup).

EXAMPLE 12

Web Server Groups :

```
<Directory /web/www.bollywood.com/movies>
    AuthName      Admin
    AuthType      Basic
    AuthUserFile  /.htpasswd
    AuthGroupFile /.htgroup
    <LIMIT GET POST>
        require  mypals
        order deny,allow
        deny from allow
        allow from bom.labs.net
    </ LIMIT>
</Directory /web/www.bollywood.com/movies>
```

- Access Is granted if a person belongs to a valid group defined in the file - **.htgroup** – and has nothing to do with the group and passwd file of login
 - Create a file **.htgroup** and put entries into it viz **mypals:foo,bar**
 - Of course, we should have added **foo** and **bar** to the **.htpasswd** file with the **htpasswd** command.
 - The **require** command requires the user to enter the **un/pwd**.
 - Access is limited to users in the group and from the domain bom.labs.net. All else is denied.
57. **AuthDBUserFile**
- The **AuthDBMUserFile** directive sets the name of a DBM file containing the list of users and passwords for user authentication. Filename is the absolute path to the user file.
Eg. **AuthDBUserFile /usr/local/.http.passwd**
 - Now to add users **foo** to this password database – of course prompting for a repeat, as usual.

```
# dbmmanage /usr/local/.http.passwd adduser foo
# dbmmanage /usr/local/.http.passwd check foo
# dbmmanage /usr/local/.http.passwd view foo
# dbmmanage /usr/local/.http.passwd update foo
# dbmmanage /usr/local/.http.passwd delete foo
```

Change **foo's** passwd

- Always use an authentication database to provide dramatic performance improvements especially if you have large web sites.
- **Security:** Make sure that the AuthDBMUserFile is stored outside the document tree of the web-server; do not put it in the directory that it protects. Otherwise, clients will be able to download the AuthDBMUserFile.

58. AuthDBGroupFile

- The **AuthDBMGroupFile** directive sets the name of a DBM file containing the list of user groups for user authentication. Filename is the absolute path to the group file.
- The group file is keyed on the username. The value for a user is a comma-separated list of the groups to which the users belongs. There must be no whitespace within the value, and it must never contain any colons.
- **Security:** make sure that the AuthDBMGroupFile is stored outside the document tree of the web-server; do not put it in the directory that it protects. Otherwise, clients will be able to download the AuthDBMGroupFile unless otherwise protected.

59. AuthDBMUserFile

- Apache HTTP Server 1.3 supported two authentication modules, **mod_auth_db** and **mod_auth_dbm**, which used **Berkeley Databases** and **DBM** databases respectively. These modules have been combined into a single module named **mod_auth_dbm** in Apache HTTP Server 2.0, which can access several different database formats.
- To migrate from mod_auth_db in version 1.3, configuration files should be adjusted by replacing **AuthDBUserFile** and **AuthDBGroupFile** with the mod_auth_dbm equivalents: **AuthDBMUserFile** and **AuthDBMGroupFile**. Also, you must add the directive **AuthDBMType DB** to indicate the type of database file in use.
- The **AuthDBMUserFile** directive sets the name of a DBM file containing the list of users and passwords for user authentication. Filename is the absolute path to the user file.
- The changes to the **.htaccess** in the doc root of the Vhost will be :

/web/www.bollywood.com/.htaccess

```
AuthType Basic
AuthName Bollywood
AuthDBMUserFile /httpasswd_dbm
AuthDBMGroupFile /dev/null
AuthDBMType DB           - Uses DBM databases
require valid-user
```

- Now to add users **foo** to this DBM password database – of course prompting for a repeat, as usual.

# htdbm -c -TDB /httpasswd_dbm foo	The -c is given only the first time.
# htdbm -m -TDB /httpasswd_dbm foo	Force MD5 encryption of the passwd. [Deprecated]
# htdbm -p -TDB /httpasswd_dbm foo	Do not encrypt the pwd. Keep it plaintext
# htdbm -s -TDB /httpasswd_dbm foo	Force SHA encrypted of the pwd
# htdbm -T -TDB /httpasswd_dbm foo	DBM Type (SDBM GDBM DB Default)
# htdbm -l -TDB /httpasswd_dbm foo	Displays the usernames from the database on stdout
# htdbm -v -TDB /httpasswd_dbm foo	Verify the username and password
# htdbm -x -TDB /httpasswd_dbm foo	Remove the username record from the DB

- This will create the file **/httpasswd_dbm** which is hashed Berkeley DB format.
- Always use an authentication database to provide dramatic performance improvements especially if you have large web sites.
- **Security:** Make sure that the AuthDBMUserFile is stored outside the document tree of the web-server; do not put it in the directory that it protects. Otherwise, clients will be able to download the AuthDBMUserFile.

60. AuthDBMGroupFile

- Apache HTTP Server 1.3 supported two authentication modules, **mod_auth_db** and **mod_auth_dbm**, which used **Berkeley Databases** and **DBM** databases respectively. These modules have been combined into a single module named **mod_auth_dbm** in Apache HTTP Server 2.0, which can access several different database formats. To migrate from mod_auth_db in version 1.3, configuration files should be adjusted by replacing **AuthDBUserFile** and **AuthDBGroupFile** with the mod_auth_dbm equivalents: **AuthDBMUserFile** and **AuthDBMGroupFile**. Also, you must add the directive **AuthDBMType DB** to indicate the type of database file in use.
- The **AuthDBMGroupFile** directive sets the name of a DBM file containing the list of user groups for user authentication. Filename is the absolute path to the group file.
- The group file is keyed on the username. The value for a user is a comma-separated list of the groups to which the users belongs. There must be no whitespace within the value, and it must never contain any colons.
- **Security:** make sure that the AuthDBMGroupFile is stored outside the document tree of the web-server; do not put it in the directory that it protects. Otherwise, clients will be able to download the AuthDBMGroupFile unless otherwise protected.

Migrating from dbmmanage [1.3] to htdbm [2.0]

- The **-m** and **-s** options work with both **dbmmanage** and **htdbm**, enabling the use of the MD5 or SHA1 algorithms for hashing passwords, respectively.
- When creating a new database with htdbm, the -c option must be used.**

Action	dbmmanage command (Apache 1.3)	Equivalent htdbm command (Apache 2.0)
Add user to database (using given password)	dbmmanage authdb add username password	htdbm -b -TDB authdb username password
Add user to database (prompts for password)	dbmmanage authdb adduser username	htdbm -TDB authdb username
Remove user from database	dbmmanage authdb delete username	htdbm -x -TDB authdb username
List users in database	dbmmanage authdb view	htdbm -l -TDB authdb
Verify a password	dbmmanage authdb check username	htdbm -v -TDB authdb username

61. **require : NOTHING OF ALL THE ABOVE AUTH DIRECTIVES WILL WORK IF THIS DIRECTIVE IS NOT PRESENT.!!**
 Actually applies the other authentication directives by saying what users will be permitted to access the specified resource. “**require**” must have **AuthName**, **AuthType**, **AuthUserFile** and **AuthGroupFile** to define the users/groups being referred to can be used in 3 ways:
- specify permitted users
 - specify permitted groups
 - state all valid users are permitted.
- Eg “**require user foo bar**”
 “foo” and “bar” should be in the **AuthUserFile** (.htpasswd)
- Eg “**require group TMS**” (specify one/more groups)
 Only users listed in the **AuthGroupFile** will be allowed access.
- Eg “**require valid-user**”
 All users listed in **AuthUserFile** will be allowed. That is, all valid linux users will be allowed.

62. **<Limit> :**

- Access controls are normally effective for all access methods, and this is the usual desired behaviour. In the general case, **access control directives should not be placed within a <limit> section.**
- The purpose of the <Limit> directive is to restrict the effect of the access controls to the nominated HTTP methods. For all other methods, the access restrictions that are enclosed in the <Limit> bracket will have no effect.

- The following example applies the access control only to the methods POST, PUT, and DELETE, leaving all other methods unprotected:

```
<Limit POST PUT DELETE>
  require valid-user
</Limit>
```

- The method names listed can be one or more of: GET, POST, PUT, DELETE, CONNECT, OPTIONS, TRACE, PATCH, PROPFIND, PROPPATCH, MKCOL, COPY, MOVE, LOCK, and **UNLOCK**.
- The method name is case-sensitive.
- If GET is used it will also restrict HEAD requests.
- The following example will prohibit the host shiva.bom.labs.net from accessing the server, but allow all other hosts.

```
<Limit GET>
  order deny,allow
  allow from all
  deny from shiva.bom.labs.net
</Limit>
```

- Another example is to restrict access to only your own domain (for an internal web server)

```
<Limit GET>
  order deny,allow
  deny from all
  allow from bom.labs.net
</Limit GET>
```

63. **<LimitExcept>** : Opposite of the **<LIMIT>** container. Directives contained in one of these sections are applied to all methods except those specified.
Eg **<LIMITExcept GET>**
Directives here
</LIMIT>
64. **FileInfo** : Whether directives that control file processing (eg **AddType**, **ErrorDocument**) will be processed if encountered in .htaccess files.
65. **Indexes** : Not to be confused with **Option → Indexes**. This controls whether the directives that affect indexing (if enabled) are processed. Such directives include "**DirectoryIndex**, **Add Description** and **Index Options**".
66. **Options** : Simply inform the server that Options directive found in .htaccess files within the scope will be processed.

EXAMPLE 13

```
<Directory /web/www.bollywood.com/movies>   Define access controls for Dir concerned
  <Limit GET POST HEAD>                     Define access methods that are being controlled
    order deny,allow
    deny from all
    allow from bom.labs.net
    allow from foo, bar
  </Limit>
</Directory>
```

This is normally given in the .htaccess file. Here we see label nesting with the Limit command.

Restricting Access to sensitive contents Using IP or Hostname

67. **Order**
These are also normally given in the **<Directory></Directory>** scoping.

68. allow	70. allow from all	72. mutual-failure
69. deny	71. deny from all	73. satisfy

67. **Order** : These specify who can get into a directory.
Defines the order in which the access rules are evaluated.
deny,allow tells httpd to apply the deny rule first and then permit exceptions to that rules based on the allow rule.

68. **allow** : This can used in 2 ways:

syntax : **allow from 'host'**

all : **allow from all or/ allow from example.com → domain or deny from Microsoft.com or allow from subnet or NW/netmask pair**

69. **deny** : Similar but opposite to above.

Order: Specifies order in which allow/deny directives are processed. Default : **deny, allow**

3 possible settings for order:

- **order deny,allow** : 'deny' processed before 'allow'. Initial state set to OK, to let everyone in so do not ask passwd.
- **order allow,deny** : All 'allow' done before 'deny'. Initial state set to FORBIDDEN, to deny access to everyone.

70. **Allow from all**

71. **Deny from all**

Eg. **Order deny,allow (BEWARE : no space allowed, only a comma)** OR else die debugging!
deny from all
allow from example.com

Deny access to everyone and then make an exception for hosts from example.com domain.

- **Mutual-failure** : See below

72. **mutual-failure** : All the "allow" AND "deny" directives MUST BE OBEYED. Only hosts that appear on the 'allow' list and don't appear on the 'deny' list are permitted access. No initial state.

73. **satisfy [any/all] –Two args** : In cases where 'require' and 'allow' or 'deny' is also used, you may need to use the '**satisfy**' directive to specify which ones need to be obeyed. Default: **all** indicates that both conditions must be fulfilled. **any** : indicates that either condition is sufficient for access. This can be used to allow some hosts to enter a protected areas without being prompted for a password but require other hosts to supply one.

MIXING HOST-BASED ACCESS CONTROL WITH HTTP AUTHENTICATION

- These entries will be in your .htaccess file which will be in the root dir of your Virtual Host

EXAMPLE 14

Permit only specific users :

```
AuthName Admin
AuthType Basic
AuthUserFile passwd/.htpasswd
AuthGroupFile /dev/null
require user foo bar
```

Only foo, bar are allowed access but have to provide valid passwds as given in **/etc/httpd/passwd/.htpasswd** bcos since no / is given file is relative to ServerRoot i.e. /etc/httpd/. Make sure you enter foo and bar into the **.htpasswd** file with the **htpasswd** command.

EXAMPLE 15

Allow Only specific users to POST :

```
AuthType Basic
AuthName DirtyDozen
AuthUserFile passwd/.htpasswd
AuthGroupFile passwd/.htgroup
< LIMIT POST>
    require group MyPals
< / LIMIT>
```

Allow anyone to view your site, but only a particular group of people will be able to post responses to your CGI program.

EXAMPLE 16

Permit / Deny Access from a Particular Domain :

```
order deny,allow
deny from all
allow from sunbeam.com
```

EXAMPLE 17

Deny access only to hosts on the **sunbeam.com** network

```
order allow, deny
allow from all
deny from sunbeam.com
```

Permit access to all except users on the sunbeam.com network

EXAMPLE 18

Protect just one file :

```
AuthType Basic
AuthName DirtyDozen
AuthUserFile passwd/.htpasswd
AuthGroupFile passwd/.htgroup
< Files admin.cgi >
    require user foo
< /File >
```

- Use a < Files > section to protect certain files but not others.

EXAMPLE 19

Block IE :

```
BrowserMatch MSIE IE
Deny from env=IE
```

Denies access to IE users

EXAMPLE 20

Using Satisfy :

```
AuthType Basic
AuthName DirtyDozen
AuthUserFile passwd/.htpasswd
AuthGroupFile passwd/.htgroup
satisfy any
allow from example.com
require group Stats
```

- Users not from a host on the **example.com** network will be prompted for a uname/passwd, but users on the **example.com** NW will be let right in.

EXAMPLE 21

AN EXAMPLE ON HTTP AUTHENTICATION

.htaccess and Password Security Access Control

- Say we have the usual default site [as a VH too] and 1 Vhosts. The details are as follows :
 - Default site : www.shiva.com
 - Doc root : `/var/www/html`
 - Make sure **ServerName** in Section 2 is set to www.shiva.com
 - Make sure you make NameVirtualHost to this site in Section 3.
 - Make sure /etc/hosts is configured to handle www.shiva.com
 - VH site : www.bollywood.com
 - Doc root : `/web/www.bollywood.com/`
 - Make sure /etc/hosts is configured to handle www.bollywood.com
 - Our default site [NameVirtualHost] is www.shiva.com and the VH is www.bollywood.com
 - Now anyone could access the default site and the VHost without uname/passwd.
 - Let's put a passwd authentication for the default site and the VH.
- 1) For default site : www.shiva.com
 Doc root : `/var/www/html`
- Create a file **.htaccess** in `/var/www/html/`

```
AuthName "Shiva's Realm"
AuthType Basic
AuthUserFile /.htpasswd
AuthUserGroup /dev/null
require valid-user
```
 - In `/etc/httpd/conf/httpd.conf`, in Section 2, you will find the following entry ::


```
<Directory /var/www/html>
```
 - Do the following anywhere in the `<Directory /var/www/html> </Directory>` container :
 - **AllowOverride AuthConfig**
 - Make sure you comment **AllowOverride None** and enable or enter the above **AllowOverride AuthConfig** or else your authentication will not be done.
 - Now enter users who you wish to access this site viz ..
- ```
htpasswd -c /.htpasswd foo (Note : foo should be a valid linux user). Change perms of /.htpasswd to 644 or else auth will never work!
Give passwd
Test whether all is OK :
```
- **Change perms of /.htpasswd to 644 or else auth will never work!**
  - `lynx www.shiva.com` ---- Will challenge for a passwd and only **foo** will be allowed
  - `telnet www.bollywood.com 80` and then type **GET /** to access the home page of [www.bollywood.com](http://www.bollywood.com)
  - Try this in IE from Windows : `http://192.168.0.10:80`
- Note :** To change **foo**'s pwd, run the command above, again, but this time without a **-c**

II ) VHost [www.bollywood.com](http://www.bollywood.com)

- Once again create a file: **.htaccess** but in **/web/www.bollywood.com/**

```
AuthName "Bollywood's Realm"
AuthType Basic
AuthUserFile /etc/passwd
AuthUserGroup /dev/null
require valid-user
```

We will be using the same **passwd** file which was created for the above default site, viz. **/etc/passwd**

- In **/etc/httpd/conf/httpd.conf**, in Section 2, make the following entry :

```
<Directory /var/www.bollywood.com>
 AllowOverride AuthConfig
</Directory>
```

- Now once you access the home page of [www.bollywood.com](http://www.bollywood.com) with **lynx** [www.bollywood.com](http://www.bollywood.com) i.e. (**/web/www.bollywood.com/index.html** page), the **.htaccess** file there will signal Apache to check for a password and prompt the user for a username/password pair.
- The other Test to see if all is working fine: **# telnet [www.bollywood.com](http://www.bollywood.com) 80** and then type **GET /** to access the home page of **www.bollywood.com**

**NOTE :-** Create **.htpasswd** in a safe place. Also, do not create it in **/root.**, since no user has access to **/root** dir and hence will not be allowed access to the file within also

#### 74. **AccessFileName .htaccess**

If "AllowOverride AuthConfig" is given in the dir scope, then the server will search for a **.htaccess** file in all subsequent subdirs. Consider this :

```
AccessFileName .control
```

Now if you are serving the file **/docs/modules/core/index.html**, the server will look for directives in the files - **.control**, **/doc/control**, **/docs/modules/control** and **/docs/modules/core/control**, before servicing the file. Directives are overridden by directives in subdirs.

- To disallow use :

```
<Directory />
 AllowOverride None
</Directory>
```

- To disallow only in portion of site :

```
<Directory /docs>
 AllowOverride None
</Directory>
```

#### 75. **DirectoryIndex** : Sets default document to display when a URL is requested in the order of the docs listed after the directive.

Eg. **DirectoryIndex index.html index.shtml /errors/no\_index.html**

- Will try each of these files ( home pages ) and if not found, will fall thru to auto indexing directive or else error that dir is forbidden !

#### 76. **UseCanonicalName [On | Off]** : If On, uses "ServerName" and "Port" directives when it constructs redirects URLs and what it passes to CGI prgs. If Off, Server will use what the client passes in. Used often on Intranet servers.

##### **CASE : ON**

- Eg. A server has a short name " www" and a directory ' splat ' viz " <http://www/splat>" – no trailing slash , then the user will have to authenticate twice – once for ' www ' and once for dir ' splat ', since apache will redirect user to '<http://www.domain.com/splat>'.

##### **CASE : OFF**

- Then apache will redirect to <http://www/splat/>.
- Of course, initially giving a 'http://www/splat/' would solve this.

#### 77. **TypesConfig /etc/mime.types** ( Multipart Internet Mail Extensions )

MIME type mappings to filename extensions

### **Content Negotiation Directives**

#### 78. **AddType** : If you have a new MIME type to add, do not edit **/etc/mime.types** but use this, which will override any similar one in effect, if given before.

Eg **AddType file/download.dnl**

#### 79. **DefaultType text/plain** : MIME type to use for files whose types cannot be determined.

Eg. You have a directory full of PostScript files that don't have filename extensions, you might use the following for that directory:

```
DefaultType application/postscript
```

80. **DefaultIcon /icons/unknown.gif**  
For a file with unknown MIME type, use this icon as default.
81. **DefaultLanguage en.en :**  
For docs not explicitly marked with a language ext. You could have a portion of your site in another language without having to add a .fr or .en ext to each file.
82. **AddLanguage en.en etc : specify language !**  
For language recognition and content negotiation.  
Eg **AddEncoding x-gzip gz tgz**  
**AddLanguage es.es**  
Now a file **myfile.gz.es** will be seen by the server as a gzipped Spanish file, **myfile.es.gz** will now be seen as the same – the right thing ! Hence files with multiple file extensions end up doing the right thing.
83. **HostNameLookups [On | Off | double ]**
- If **Off**, only IPs will appear in logs.
  - If **On**, a DNS lookup is done for every doc that client requests, which slows things down – and hostnames **and also** IPs are now written to logs.
  - **Double** performs reverse lookups – reverse + forward (called **PARANOID** in tcpwrapper terminology)
84. **Redirect :** When a webpage is moved, Redirect can be used to map the old URL to a new URL. The format is as follows:

Eg : **Redirect /path/foo.html http://new\_domain/path/foo.html**

So, if an HTTP request is received for a page which used to be found at `http://your_domain/path/foo.html`, the server will send back the new URL (`http://new_domain/path/foo.html`) to the client, which should attempt to fetch the document from the new URL.

## HTTP as a Proxy Server

### Why a Proxy Server?

- A cache is a locally maintained copy of a remote server's Web page. Servers that cache Web pages are called **Proxy** servers.
- Most Proxies are used along with firewalls, direct access to the Internet is often blocked. Users connect to the proxy server thru the local network [**black IP addrs**] and the proxy server is trusted to connect to the Internet with a **white black addr [valid on the Internet – static]**. Proxy servers do not **have** to maintain cached copies of web pages, but caching improves performance by reducing the amt of traffic sent over the WAN and by reducing contention for popular web sites.

### What is a Web Cache?

- Pages relayed thru a server are saved in the **CacheRoot** is **/var/cache/httpd**. This is called **Caching**. If a page is available in the cache it is served to the client and there is no need to go further. This cuts down access time significantly [30%]. Especially if you make huge disk caches, increase Memory etc.

### PROS:

- Single Point of entry hence better monitoring
- Logging of traffic and hence can determine bandwidth usage
- Can check which sites visited by Clients
- Can block unwanted sites and traffic eg banner sites to speed up throughput

### CONS:

- Single Point of Failure for Internet Traffic – Use backup Proxies
- Slows down speed of access
- Content negotiation is not cacheable
- Many Web objects, like streaming Video/Audio Content negotiation is not cacheable or do not cache well.
- Anything that is authenticated is not cached
- Objects obtained by a secure protocol are not cached.

### VERDICT:

- APACHE is the best Web Server in the World and should stick to doing just that.
- BUT SQUID is a better and faster solution for caching and proxying.
- The following **10** directives come into play in setting up Apache as a Proxy server :

85. **ProxyRequests**  
86. **ProxyRemote**  
87. **ProxyPass**  
88. **ProxyPassReverse**  
89. **ProxyBlock**  
90. **AllowCONNECT**  
91. **ProxyReceiveBufferSize**  
92. **NoProxy**  
93. **ProxyDomain**  
94. **ProxyVia**

### 85. ProxyRequests

- This allows or prevents Apache from functioning as a proxy server. **Def: No**. If you set it to yes, you enable Apache as a Proxy server.
- If you uncomment the **IfModule** tags surrounding the **ProxyRequests** directives, your Apache HTTP Server will also function as a proxy server. You will also need to load the **mod\_proxy** module.

- Setting **ProxyRequests** to 'off' does not disable use of the **ProxyPass** directive.

#### 86. **ProxyRemote**

**Syntax:** ProxyRemote <match> <remote-server>

- On a big WAN, with many many LANS, each LAN could have a Proxy. Hence each Proxy becomes a remote Proxy for another LAN. This is called a **Cache Hierarchy**.
- If you have one such hierarchy, then you might want to fetch the object from another cache within the hierarchy.
- With this directive you can configure how each cache server should communicate with the other proxies within the cache hierarchy.
- **Match** - partial URL that the server will try to match with each incoming proxy request. If the first part of the RUL is matched then the request will be fwded to the proxy specified by the the remote-server parameter.

#### EXAMPLE 22

**ProxyRemote** <http://goodguys.com/> <http://mirrorquys.com:8000>

- This cmd is triggered once a request arrives at the proxy server, containing a URL addressed to <http://goodguys.com/>. The proxy will try to access the requested page by sending another proxy request thru another proxy server <http://mirrorquys.com:8000> which is listening on Port 8080. When the requested resource is returned, its fwded to the Client that requested it.

#### EXAMPLE 23

**ProxyRemote** \* <http://cleversite.com>:8080

- All incoming proxy requests will be fwded to and served by a server named <http://cleversite.com/> that's supplying proxy services on Port 8080.

#### EXAMPLE 24

**ProxyRemote** ftp <http://anotherproxy:8080>

- The proxy will take an incoming FTP request, encapsulated as yet another HTTP proxy request, that is using the HTTP protocol, try to fetch the requested resource from another proxy server.

#### 87. **ProxyPass**

**Syntax :** ProxyPass <path> <url>

- This directive allows remote servers to be mapped into the space of the local server; the local server does not act as a proxy in the conventional sense, but appears to be a mirror of the remote server. <path> is the name of a local virtual path; <url> is a partial URL for the remote server.

#### EXAMPLE 25

- Suppose a local server has address <http://wibble.org/>; then

**ProxyPass** /mirror/fool <http://foo.com/>

will cause a local request for the <<http://wibble.org/mirror/fool/bar>> to be internally converted into a proxy request to <<http://foo.com/bar>>.

- It can be used to good effect to fetch a resource from an intranet and deliver it unto the Internet. For all outward appearances, the material is being delivered from your Internet Server.

#### 88. **ProxyPassReverse**

**Syntax:** ProxyPassReverse <path> <url>

- This directive lets Apache adjust the URL in the Location header on HTTP redirect responses. For instance this is essential when Apache is used as a reverse proxy to avoid by-passing the reverse proxy because of HTTP redirects on the backend servers which stay behind the reverse proxy. <path> is the name of a local virtual path. <url> is a partial URL for the remote server - the same way they are used for the ProxyPass directive.

#### EXAMPLE 26

- Let's expand the last example .

- Suppose a local server has address <http://wibble.org/>; then

**ProxyPass** /mirror/fool <http://foo.com/>

**ProxyPassReverse** /mirror/fool <http://foo.com/>

will not only cause a local request for the <<http://wibble.org/mirror/fool/bar>> to be internally converted into a proxy request to <<http://foo.com/bar>> (the functionality ProxyPass provides here).

- It also takes care of redirects the server **foo.com** sends: when <http://foo.com/bar> is redirected by him to <http://foo.com/quux>, Apache adjusts this to <http://wibble.org/mirror/fool/quux> before forwarding the HTTP redirect response to the client.

#### 89. **ProxyBlock**

**Syntax:** ProxyBlock <word/host/domain list>

- The ProxyBlock directive specifies a list of words, hosts and/or domains, separated by spaces. HTTP, HTTPS, and FTP document requests to sites whose names contain matched words, hosts or domains are blocked by the proxy server.
- The proxy module will also attempt to determine IP addresses of list items which may be hostnames during startup, and cache them for match test as well.

#### EXAMPLE 27

- Check out this example :

**ProxyBlock** a.joes-garage.com some-host.co.uk rocky.wotsamattau.edu

- 'rocky.wotsamattau.edu' would also be matched if referenced by IP address since it is a host..
- Note that 'wotsamattau' would also be sufficient to match 'wotsamattau.edu'.
- Note also that

**ProxyBlock** \*

blocks connections to all sites.

90. **AllowCONNECT****Syntax:** AllowCONNECT <port list>**Default:** AllowCONNECT 443 563

- The AllowCONNECT directive specifies a list of port numbers to which the proxy CONNECT method may connect. Today's browsers use this method when a **https** connection is requested and proxy tunneling over http is in effect.
- By default, only the default **https port (443)** and the default **snews port (563)** are enabled.
- Use the AllowCONNECT directive to override this default and allow connections to the listed ports only.

91. **ProxyReceiveBufferSize****Syntax:** ProxyReceiveBufferSize <bytes>

- The **ProxyReceiveBufferSize** directive specifies an explicit network buffer size for outgoing HTTP and FTP connections, for increased throughput.
- It has to be greater than 512 or set to 0 to indicate that the system's default buffer size should be used.

**EXAMPLE 28****ProxyReceiveBufferSize 2048**92. **NoProxy****Syntax:** NoProxy { <Domain> | <SubNet> | <IpAddr> | <Hostname> }

- We saw before that on our WAN, there could be many a LAN proxy.
- There are times when we wish to see our requests served asap, w/o being fwded to servers defined with the **ProxyRemote** directive.
- Most often this will be the case for requests to servers located on your intranet
- Hence, this directive is only useful for Apache remote proxy servers within intranets.
- The **NoProxy** directive specifies a list of subnets, IP addresses, hosts and/or domains, separated by spaces. A request to a host which matches one or more of these is always served directly, without forwarding to the configured ProxyRemote proxy server(s).

**EXAMPLE 29**

```
ProxyRemote * http://firewall.mycompany.com:81
NoProxy .mycompany.com 192.168.112.0/21
```

The arguments to the NoProxy directive are one of the following type list:

**Domain :**

- A Domain is a partially qualified DNS domain name, preceded by a period. It represents a list of hosts which logically belong to the same DNS domain or zone (i.e., the suffixes of the hostnames are all ending in Domain).
- **Example:** .com .apache.org.
- To distinguish Domains from Hostnames (both syntactically and semantically; a DNS domain can have a DNS A record, too!), Domains are always written with a leading period.
- **Note:** Domain name comparisons are done without regard to the case, and Domains are always assumed to be anchored in the root of the DNS tree, therefore two domains **.MyDomain.com** and **.mydomain.com**. (note the trailing period) are considered equal. Since a domain comparison does not involve a DNS lookup, it is much more efficient than subnet comparison.

**SubNet**

- A SubNet is a partially qualified internet address in numeric (dotted quad) form, optionally followed by a slash and the netmask, specified as the number of significant bits in the SubNet. It is used to represent a subnet of hosts which can be reached over a common network interface. In the absence of the explicit net mask it is assumed that omitted (or zero valued) trailing digits specify the mask. (In this case, the netmask can only be multiples of 8 bits wide.)

**Examples:** 192.168 or 192.168.0.0

- the subnet 192.168.0.0 with an implied netmask of 16 valid bits (sometimes used in the netmask form 255.255.0.0) 192.168.112.0/21 the subnet 192.168.112.0/21 with a netmask of 21 valid bits (also used in the form 255.255.248.0)
- As a degenerate case, a SubNet with 32 valid bits is the equivalent to an IPAddr, while a SubNet with zero valid bits (e.g., 0.0.0.0/0) is the same as the constant `_Default_`, matching any IP address.

**IPAddr**

- A IPAddr represents a fully qualified internet address in numeric (dotted quad) form. Usually, this address represents a host, but there need not necessarily be a DNS domain name connected with the address.

**Example:** 192.168.123.7

- **Note:** An IPAddr does not need to be resolved by the DNS system, so it can result in more effective Apache performance.

**Hostname**

- A Hostname is a fully qualified DNS domain name which can be resolved to one or more IPAddrs via the DNS domain name service. It represents a logical host (in contrast to Domains, see above) and must be resolvable to at least one IPAddr (or often to a list of hosts with different IPAddr's).

**Examples:** prep.ai.mit.edu www.apache.org.

- **Note:** In many situations, it is more effective to specify an IPAddr in place of a Hostname since a DNS lookup can be avoided. Name resolution in Apache can take a remarkable deal of time when the connection to the name server uses a slow PPP link.
- **Note:** Hostname comparisons are done without regard to the case, and Hostnames are always assumed to be anchored in the root of the DNS tree, therefore two hosts **WWW.MyDomain.com** and **www.mydomain.com**. (note the trailing period) are considered equal.



93. **ProxyDomain**

**Syntax:** ProxyDomain <Domain>

- This directive is only useful for Apache proxy servers within intranets.
- Users often leave out the domain part of hosts that are part of the intranet.
- With the **ProxyDomain** directive you can specify which domain should be appended to hostnames that are not fully qualified.
- When the server receives a request w/o a FQDN, it will try to redirect the request to a host within the specified domain

**EXAMPLE 30**

```
ProxyRemote * http://firewall.mycompany.com:81
NoProxy .mycompany.com 192.168.112.0/21
ProxyDomain .mycompany.com
```

- Here, assuming the user has typed – <http://www.index.html>, the following directive above would tell the server to redirect the response to <http://www.mycompany.com/index.html>

94. **ProxyVia**

**Syntax:** ProxyVia {off | on | full | block}

**Default:** ProxyVia off

- This directive controls the use of the **Via:** HTTP header by the proxy. Its intended use is to control the flow of proxy requests along a chain of proxy servers. That is the flow of proxy requests with proxy hierarchies
- **OFF:** Default, prevents processing of Via: headers, and requests/replies are passed through unchanged.
- **ON:** If set to on, each request and reply will get [added] a Via: header line added for the current host.
- **FULL:** If set to full, each generated Via: header line will additionally have the Apache server version shown as a Via: comment field.
- **Block:** If set to block, every proxy request will have all its Via: header lines removed. No new Via: header will be generated.

**HTTP as a Caching Server**

**Cache Directives :** A number of cache directives are commented out in the proxy IfModule tags mentioned above. If you are using the proxy server functionality and you want to also enable the proxy cache, you should uncomment the cache directives as described. The default settings for your cache directives should be appropriate for most configurations.

- The following 11 directives come into play in setting up Apache as a Caching server :

```
95 CacheRoot
96 CacheSize
97 CacheGcInterval
98 CacheMaxExpire
99 CacheLastModifiedFactor
100 CacheDefaultExpire
101 NoCache
102 CacheDirLength
103 CacheDirLevels
104 CacheForceCompletion
105 CacheNegotiatedDocs
```

- 95. **CacheRoot** Sets the name of the directory to contain cache files; this must be /readablewritable by the httpd server. (see the **User** directive).

- **Setting CacheRoot enables Cacheing; without defining a CacheRoot, no caching is enabled.**
- **If ProxyRequests is set, then Proxy functionality will be available, but then no Cacheing will be available.**
- **Hence, to make Apache into a [HTTP-Caching and Proxy Server both ProxyRequests and CacheRoot must be set]**
- **CacheRoot** sets the name of the directory which will contain cached files [web pages]. The default **CacheRoot** is **/var/cache/httpd/**  
**CacheRoot /var/cache/httpd**

- 96. **CacheSize** sets how much disk space the cache can use, in KB. The default CacheSize is 5 KB. Depending on the expected proxy traffic volume and **CacheGcInterval**, use a value which is at least 20 to 40 % lower than the available space in the partition or else it could fill up if garbage collection is not enabled [pruning]

**CacheRoot 5**

- 97. **CacheGcInterval** : sets a number of hours. After that number of hours, files in the cache will be pruned if the cache is using more space than allowed by **CacheSize**. There is no default for CacheGcInterval hence see that you make it some default like 4 hours, if, of course, caching is enabled.

**CacheGcInterval 1.5**

- This would check the cache every 90 mins, and delete files if the space usage is greater than that set by **CacheSize**.
- **Note** that <time> accepts a float value, hence we set it to a float value 1.5 i.e. 90 mins.
- If unset, **no garbage collection will be performed**, and the cache will grow indefinitely.
- **Note** also that the larger the CacheGcInterval, the more extra space beyond the configured CacheSize will be needed for the cache between garbage collections.

- 98. **CacheMaxExpire** : Many objects have their expiry dates set by their originating servers. These cached objects may not have expiry dates in some cases. How often should a server check for these expiry dates is what this directive specifies. By default it checks the expiry dates of cached HTML documents after every 24 hrs. If expired, then the object will be deleted or else retained (without a reload from the originating Web server) in the cache. The max number of hours to do a check is set by **CacheMaxExpire**. If the expiry date has not yet expired on a check, the object is untouched and allowed to cache. The default is 24 hours.

**CacheMaxExpire 1.5**

- This 1.5 would indicate that all objects should be checked at least every 90 mins for their expiry dates.

99. **CacheLastModifiedFactor** This directive is for docs [objects] in the cache which do not have an expiry date set. Then how does one know when to delete this object from the cache?  
By using a fake expiry date.

**CacheLastModifiedFactor 0.1**

- First, httpd checks the cached object for its last modification value. This is written in the object by the originating server.
- Of course, this object would have no expiry date.
- This value is multiplied by the **factor – 0.1** – to calculate the **fake expiry date**. It uses the formula below. The default factor is 0.1]  
**Fake expiry-period = time-since-last-modification \* <factor>**

Therefore, if a doc is retrieved that was modified 10 hrs ago, it is only held in the cache for 1 hr before a fresh copy is requested. The assumption is that if a doc changes frequently, the time of its last modification will be recent. Thus docs that change frequently are cached only a short period of time.

Regardless, nothing is cached longer than **CacheMaxExpire**

100. **CacheDefaultExpire** is the expiry time in hours for a document that was received using a protocol that does not support expiry times at all. Hence even the above calculation would be useless since, anyway, whatever expiration date is calculated, the protocol would just not support expiry dates. This value is set to a default of 1 hour.  
**CacheMaxExpire** does not override this setting.

**CacheDefaultExpire 1**

101. **NoCache** Any document that is retrieved from a host or domain that matches one set in **NoCache** will not be cached. If you know of hosts or domains from which you do not want to cache documents, uncomment NoCache and set their domains or hostnames here.

**NoCache a.org server.somewhere.org**

Here, requests to hosts within the **a.org** domain are not cached nor are any requests to host **server.somewhere.org** which is a single machine. Since it is a host, Apache will try to determine its IP addr and any request that matches that address won't be cached, either.

102. **CacheDirLength** Used to set the no of chars in the names of subdirs in the cache. **Def: 1**

103. **CacheDirLevels** Used to set the no of subdirs levels in the cache. [Below **CacheRoot**] **Def: 3**

104. **CacheForcCompletion** : If a transferring request is cancelled abruptly [**connection reset by peer**], caching whatever was transferred is always a good idea. You can set how much [**Percentage**] of a web object will have to be transferred to consider a request to be completed. Even if a Client requests cancellation. By hitting the Stop button on the Browser.

**CacheForcCompletion 90**

This is a percentage, and must be a number between 1 and 100, or **0 to use the default**. 100 will cause a document to be cached only if the transfer was allowed to complete fully. A number between 60 and 90 is recommended.

105. **CacheNegotiatedDocs** : If set, this directive allows content-negotiated documents to be cached by proxy servers. This could mean that clients behind those proxys could retrieve versions of the documents that are not the best match for their abilities, but it will make caching more efficient. This directive only applies to requests which come from HTTP/1.0 browsers. HTTP/1.1 provides much better control over the caching of negotiated documents, and this directive has no effect in responses to HTTP/1.1 requests.

106. **UserDir**

**Syntax:** **UserDir** directory/filename

**Default:** **UserDir** public\_html

- **UserDir** is the name of the subdirectory within each user's home directory where they should place personal HTML files which are to be served by the Web server.
- That is, **UserDir** directive sets the name of the directory that is appended onto a user's home directory when a request for a document for a user is received.
- If neither the enabled nor the disabled keywords appear in the **Userdir** directive, the argument is treated as a filename pattern, and is used to turn the name into a directory specification
- By default, the subdirectory is **public\_html**

### EXAMPLE 31

**UserDir** public\_html

- A users enter the following URL in a Browser : <http://www.someserver.com/~foo/somepage.html>
- After translation the server would look for the foll file : **/home/foo/public\_html/somepage.html**
- The User will be served the following file : **/home/foo/public\_html/somepage.html**
- **Note** : The default path to users' home directories may be different on your system).
- Make sure that the permissions on the users' home directories are set correctly. Users' home directories must be set to 0711. The read (r) and execute (x) bits must be set on the users' **public\_html** directories (0755 will also work).
- Now why did I say also?
- Files that will be served in users' public\_html directories must be set to at least 0644.
- This directive is set to disable by default i.e. **UserDir** **disable**

### EXAMPLE 32

**UserDir** /usr/web

- A users enter the following URL in a Browser : <http://www.someserver.com/~foo/one/two.html>
- This will be translated to : **/usr/web/foo/public\_html/one/two.html**
- He will be served the following file : **/home/foo/public\_html/one/two.html**

**EXAMPLE 33****UserDir htmldocs**

- A user enters the following URL in a Browser : <http://www.someserver.com/~foo/test.html>
- He will be served the following file : </home/foo/htmldocs/test.html>

**EXAMPLE 34**

- If your user's pages are housed on an entirely different server, you can redirect to a URL, using the asterisk as a placeholder for a username

**UserDir http://users.linux.net/\*/**

- A user enters the following URL in a Browser : <http://www.linux.net/~rbowen/test.html>
- He will be mapped to the full URL : <http://users.linux.net/rbowen/test.html>
- If neither the enabled nor the disabled keywords appear in the Userdir directive, the argument is treated as a filename pattern, and is used to turn the name into a directory specification. A request for <http://www.foo.com/~bob/one/two.html> will be translated to:
 

```
UserDir public_html -> ~bob/public_html/one/two.html
UserDir /usr/web -> /usr/web/bob/one/two.html
UserDir /home/*/*www -> /home/bob/www/one/two.html
```
- The following directives will send redirects to the client:
 

```
UserDir http://www.foo.com/users -> http://www.foo.com/users/bob/one/two.html
UserDir http://www.foo.com/*/*usr -> http://www.foo.com/bob/usr/one/two.html
UserDir http://www.foo.com/~*/ -> http://www.foo.com/~bob/one/two.html
```
- Be careful when using this directive; for instance, "UserDir ./" would map "/~root" to "/" - which is probably undesirable. If you are running Apache 1.3 or above, it is strongly recommended that your configuration include a "UserDir disabled root" declaration. See also the <Directory> directive.

**EXAMPLE-EXERCISE 35 Hosting in Virtual Directory**

- Why is it needed?
- We know that <http://shiva.bom.labs.net> or <http://127.0.0.1/> is mapped with </var/www/html/index.html>
- And if the file is not **index.html** let's say **a.html**
- Then we have to call <http://shiva.bom.labs.net/a.html>
- Now suppose a client of yours does not want to write the file name after the URL.
- In that case the web server says "create a login for each user who wants to host his site and then make a directory called "public\_html" under login directory and then put index.html under that directory"
- For that the directive in </etc/httpd/conf/httpd.conf> should look like.

**UserDir public\_html**

```
<Directory /home/*/*public_html>
 AllowOverride FileInfo AuthConfig Limit
 Options MultiViews Indexes SymLinksIfOwnerMatch IncludesNoExec
 <Limit GET POST OPTIONS PROPFIND>
 Order allow,deny
 Allow from all
 </Limit>
 <Limit PUT DELETE PATCH PROPPATCH MKCOL COPY MOVE LOCK UNLOCK>
 Order deny,allow
 Deny from all
 </Limit>
</Directory>
```

- Login as User **foo**, where **foo** is a valid Linux user and login
- `$ mkdir public_html`
- `$ cd public_html`
- `$ vi index.html`
- And enter the code
 

```
<html>
<h1> see the virtual login hosting</h1>
 <form method=post action=/cgi-bin/b.pl>
 <input type=submit>
 </form>
</html>
```
- `$ cd /etc/httpd/cgi-bin/`
- `$ vi b.pl`
- And enter the following code:
 

```
#!/usr/bin/perl
print "Content-type:text/html \n\n";
print "<h1>Welcome in foo's Virtual Site</h1>";
print "you have accessed through $ENV{REQUEST_METHOD}";
print "<h2>hope u visit again</h2>";
```
- `$ lynx http://shiva.bom.labs.net/~foo/` OR `$ lynx http://192.168.0.10/~foo/` and you will be shown the page [/home/foo/public\\_html/index.html](/home/foo/public_html/index.html)
- Do it for other logins also.

107. **ServerAlias****EXAMPLE 36**

- If your server had more one name, you can use ServerAlias in your httpd.conf

```
<VirtualHost shiva.bom.labs.net>
....
 ServerName shiva.bom.labs.net
 ServerAlias www.shiva.bom.labs.net
 ServerAlias www.siva.bom.labs.net
...
</VirtualHost>
```

**HTTP Logging**108. **ErrorLog****Default:** ErrorLog logs/error\_log

- The error log directive sets the name of the file to which the server will log any errors it encounters. If the filename does not begin with a slash (/) then it is assumed to be relative to the ServerRoot. If the filename begins with a pipe (|) then it is assumed to be a command to spawn to handle the error log.

109. **LogLevel****Syntax:** LogLevel level**Default:** LogLevel error

- LogLevel adjusts the verbosity of the messages recorded in the error logs (see ErrorLog directive). The following levels are available, in order of decreasing significance:

**Level Description****Example****emerg** Emergencies - system is unusable.

"Child cannot open lock file. Exiting"

**alert** Action must be taken immediately.

"getpwuid: couldn't determine user name from uid"

**crit** Critical Conditions.

"socket: Failed to get a socket, exiting child"

**error** Error conditions.

"Premature end of script headers"

**warn** Warning conditions.

"child process 1234 did not exit, sending another SIGHUP"

**notice** Normal but significant condition.

"httpd: caught SIGBUS, attempting to dump core in ..."

**info** Informational.

"Server seems busy, (you may need to increase StartServers, or Min/MaxSpareServers)..."

**debug** Debug-level messages

"Opening config file ..."

- When a particular level is specified, messages from all other levels of higher significance will be reported as well. E.g., when LogLevel info is specified, then messages with log levels of notice and warn will also be posted.
- Using a level of at least crit is recommended.

110. **LogFormat****Syntax:** LogFormat *format* [*nickname*]**Default:** LogFormat "%h %l %u %t \"%r\" %s %b"**Context:** server config, virtual host

- This sets the format of the default logfile named by the **TransferLog** directive .
- If you include a nickname for the format on the directive line, you can use it in other **LogFormat** and **CustomLog** directives rather than repeating the entire format string.
- A LogFormat directive which defines a nickname does nothing else -- that is, it only defines the nickname, it doesn't actually apply the format and make it the default.

## Custom Log Formats

- The format argument to the **LogFormat** and **CustomLog** is a string. This string is logged to the log file for each request. It can contain literal characters copied into the log files, and `'%'` directives which are replaced in the log file by the values as follows:

`%...a:` Remote IP-address  
`%...A:` Local IP-address  
`%...B:` Bytes sent, excluding HTTP headers.  
`%...b:` Bytes sent, excluding HTTP headers. In CLF format i.e. a '-' rather than a 0 when no bytes are sent.  
`%...{FOOBAR}e:` The contents of the environment variable FOOBAR  
`%...f:` Filename  
`%...h:` Remote host  
`%...H:` The request protocol  
`%...{Foobar}i:` The contents of Foobar: header line(s) in the request sent to the server.  
`%...l:` Remote logname (from identd, if supplied)  
`%...m:` The request method  
`%...{Foobar}n:` The contents of note "Foobar" from another module.  
`%...{Foobar}o:` The contents of Foobar: header line(s) in the reply.  
`%...p:` The canonical Port of the server serving the request  
`%...P:` The process ID of the child that serviced the request.  
`%...q:` The query string (prepended with a ? if a query string exists, otherwise an empty string)  
`%...r:` First line of request  
`%...s:` Status. For requests that got internally redirected, this is the status of the \*original\* request --- `%...>s` for the last.  
`%...t:` Time, in common log format time format (standard english format)  
`%...{format}t:` The time, in the form given by format, which should be in `strftime(3)` format. (potentially localised)  
`%...T:` The time taken to serve the request, in seconds.  
`%...u:` Remote user (from auth; may be bogus if return status (%s) is 401)  
`%...U:` The URL path requested.  
`%...v:` The canonical ServerName of the server serving the request.  
`%...V:` The server name according to the UseCanonicalName setting.

- The `'...'` can be nothing at all (e.g., `"%h %u %r %s %b"`), or it can indicate conditions for inclusion of the item (which will cause it to be replaced with `'-'` if the condition is not met). Note that there is no escaping performed on the strings from `%r`, `%...i` and `%...o`; some with long memories may remember that I thought this was a bad idea, once upon a time, and I'm still not comfortable with it, but it is difficult to see how to 'do the right thing' with all of `'%...i'`, unless we URL-escape everything and break with CLF.
- The forms of condition are a list of HTTP status codes, which may or may not be preceded by `'!'`. Thus, `'%400,501{User-agent}i'` logs User-agent: on 400 errors and 501 errors (Bad Request, Not Implemented) only; `'%!200,304,302{Referer}i'` logs Referer: on all requests which did not return some sort of normal status.
- Note that the common log format is defined by the string `"%h %l %u %t \"%r\" %s %b"`, which can be used as the basis for extending for format if desired (e.g., to add extra fields at the end). NCSA's extended/combined log format would be `"%h %l %u %t \"%r\" %s %b \"%{Referer}i\" \"%{User-agent}i\""`.
- Note that the canonical ServerName and Port of the server serving the request are used for `%v` and `%p` respectively. This happens regardless of the UseCanonicalName setting because otherwise log analysis programs would have to duplicate the entire vhost matching algorithm in order to decide what host really served the request.

### 111. **TransferLog**

#### **Syntax: TransferLog file-pipe**

#### **Default: none**

- The TransferLog directive adds a log file in the format defined by the most recent LogFormat directive, or Common Log Format if no other default format has been specified.
- File-pipe** is one of
  - A filename : A filename relative to the ServerRoot.
  - | : followed by a command
    - A program to receive the agent log information on its standard input. Note the a new program will not be started for a VirtualHost if it inherits the TransferLog from the main server.
- Security: if a program is used, then it will be run under the user who started httpd. This will be root if the server was started by root; be sure that the program is secure.

\*\*\*\*\* **End of Directive Section** \*\*\*\*\*

## **/etc/httpd/conf/httpd.conf - The Apache Configuration File**

### **### Section 1: Global Environment**

```
ServerTokens OS
ServerRoot "/etc/httpd"
#ScoreBoardFile run/httpd.scoreboard
PidFile run/httpd.pid
Timeout 300
KeepAlive Off
MaxKeepAliveRequests 100
KeepAliveTimeout 15
```

### **## Server-Pool Size Regulation (MPM specific)**

```
prefork MPM
StartServers: number of server processes to start
MinSpareServers: minimum number of server processes which are kept spare
MaxSpareServers: maximum number of server processes which are kept spare
MaxClients: maximum number of server processes allowed to start
MaxRequestsPerChild: maximum number of requests a server process serves
```

```
<IfModule prefork.c>
 StartServers 8
 MinSpareServers 5
 MaxSpareServers 20
 MaxClients 150
 MaxRequestsPerChild 1000
</IfModule>
```

```
worker MPM
StartServers: initial number of server processes to start
MaxClients: maximum number of simultaneous client connections
MinSpareThreads: minimum number of worker threads which are kept spare
MaxSpareThreads: maximum number of worker threads which are kept spare
ThreadsPerChild: constant number of worker threads in each server process
MaxRequestsPerChild: maximum number of requests a server process serves
```

```
<IfModule worker.c>
 StartServers 2
 MaxClients 150
 MinSpareThreads 25
 MaxSpareThreads 75
 ThreadsPerChild 25
 MaxRequestsPerChild 0
</IfModule>
```

```
perchild MPM
NumServers: constant number of server processes
StartThreads: initial number of worker threads in each server process
MinSpareThreads: minimum number of worker threads which are kept spare
MaxSpareThreads: maximum number of worker threads which are kept spare
MaxThreadsPerChild: maximum number of worker threads in each server process
MaxRequestsPerChild: maximum number of connections per server process
```

```
<IfModule perchild.c>
 NumServers 5
 StartThreads 5
 MinSpareThreads 5
 MaxSpareThreads 10
 MaxThreadsPerChild 20
 MaxRequestsPerChild 0
</IfModule>
```

```
Listen 80
Include conf.d/*.conf
```

### **# Dynamic Shared Object (DSO) Support**

```
LoadModule access_module modules/mod_access.so
LoadModule auth_module modules/mod_auth.so
LoadModule auth_anon_module modules/mod_auth_anon.so
LoadModule auth_dbm_module modules/mod_auth_dbm.so
LoadModule auth_digest_module modules/mod_auth_digest.so
LoadModule include_module modules/mod_include.so
LoadModule log_config_module modules/mod_log_config.so
LoadModule env_module modules/mod_env.so
```

```

LoadModule mime_magic_module modules/mod_mime_magic.so
LoadModule cern_meta_module modules/mod_cern_meta.so
LoadModule expires_module modules/mod_expires.so
LoadModule headers_module modules/mod_headers.so
LoadModule usertrack_module modules/mod_usertrack.so
LoadModule unique_id_module modules/mod_unique_id.so
LoadModule setenvif_module modules/mod_setenvif.so
LoadModule mime_module modules/mod_mime.so
LoadModule dav_module modules/mod_dav.so
LoadModule status_module modules/mod_status.so
LoadModule autoindex_module modules/mod_autoindex.so
LoadModule asis_module modules/mod_asis.so
LoadModule info_module modules/mod_info.so
LoadModule cgi_module modules/mod_cgi.so
LoadModule dav_fs_module modules/mod_dav_fs.so
LoadModule vhost_alias_module modules/mod_vhost_alias.so
LoadModule negotiation_module modules/mod_negotiation.so
LoadModule dir_module modules/mod_dir.so
LoadModule imap_module modules/mod_imap.so
LoadModule actions_module modules/mod_actions.so
LoadModule speling_module modules/mod_speling.so
LoadModule userdir_module modules/mod_userdir.so
LoadModule alias_module modules/mod_alias.so
LoadModule rewrite_module modules/mod_rewrite.so
LoadModule proxy_module modules/mod_proxy.so
LoadModule proxy_ftp_module modules/mod_proxy_ftp.so
LoadModule proxy_http_module modules/mod_proxy_http.so
LoadModule proxy_connect_module modules/mod_proxy_connect.so
#ExtendedStatus On

```

### ### Section 2: 'Main' server configuration

```

User apache
Group apache
ServerAdmin root@shiva.bom.labs.net
ServerName shiva.bom.labs.net:80
UseCanonicalName Off
DocumentRoot "/var/www/html"
<Directory />
 Options FollowSymLinks
 AllowOverride None
</Directory>
<Directory "/var/www/html">
 Options Indexes FollowSymLinks
 AllowOverride Authconfig
 Order allow,deny
 Allow from all
</Directory>
<Directory /web/www.bollywood.com>
 AllowOverride AuthConfig
</Directory>

<LocationMatch "^/$">
 Options -Indexes
 ErrorDocument 403 /error/noindex.html
</LocationMatch>

<IfModule mod_userdir.c>
 UserDir disable
 #UserDir public_html
</IfModule>

#<Directory /home/*/public_html>
AllowOverride FileInfo AuthConfig Limit
Options MultiViews Indexes SymLinksIfOwnerMatch IncludesNoExec
<Limit GET POST OPTIONS>
Order allow,deny
Allow from all
</Limit>
<LimitExcept GET POST OPTIONS>
Order deny,allow
Deny from all
</LimitExcept>
#</Directory>

```



```

DirectoryIndex index.html index.html.var

AccessFileName .htaccess

<Files ~ "^\.ht">
 Order allow,deny
 Deny from all
</Files>

<Files *.php>
 SetOutputFilter PHP
 SetInputFilter PHP
</Files>
TypesConfig /etc/mime.types

DefaultType text/plain

<IfModule mod_mime_magic.c>
MIMEMagicFile /usr/share/magic.mime
 MIMEMagicFile conf/magic
</IfModule>

HostnameLookups Off

ErrorLog logs/error_log

LogLevel warn

LogFormat "%h %l %u %t \"%r\" %>s %b \"%{Referer}i\" \"%{User-Agent}i\"" combined
LogFormat "%h %l %u %t \"%r\" %>s %b" common
LogFormat "%{Referer}i -> %U" referer
LogFormat "%{User-agent}i" agent

CustomLog logs/access_log combined

#CustomLog logs/referer_log referer
#CustomLog logs/agent_log agent

#CustomLog logs/access_log combined

ServerSignature On

Alias /icons/ "/var/www/icons/"

<Directory "/var/www/icons">
 Options Indexes MultiViews
 AllowOverride None
 Order allow,deny
 Allow from all
</Directory>

Alias /manual "/var/www/manual"

<Directory "/var/www/manual">
 Options Indexes FollowSymLinks MultiViews
 AllowOverride None
 Order allow,deny
 Allow from all
</Directory>

<IfModule mod_dav_fs.c>
Location of the WebDAV lock database.
 DAVLockDB /var/lib/dav/lockdb
</IfModule>

ScriptAlias /cgi-bin/ "/var/www/cgi-bin/"

<IfModule mod_cgid.c>
#Scriptsock logs/cgisock
</IfModule>

```

```

<Directory "/var/www/cgi-bin">
 AllowOverride None
 Options None
 Order allow,deny
 Allow from all
</Directory>

Redirect permanent /foo http://www.example.com/bar

IndexOptions FancyIndexing VersionSort NameWidth=*

AddIconByEncoding (CMP,/icons/compressed.gif) x-compress x-gzip

AddIconByType (TXT,/icons/text.gif) text/*
AddIconByType (IMG,/icons/image2.gif) image/*
AddIconByType (SND,/icons/sound2.gif) audio/*
AddIconByType (VID,/icons/movie.gif) video/*

AddIcon /icons/binary.gif .bin .exe
. snip

DefaultIcon /icons/unknown.gif

ReadmeName README.html
HeaderName HEADER.html

IndexIgnore .??* *~ *# HEADER* README* RCS CVS *,v *,t

AddEncoding x-compress Z
AddEncoding x-gzip gz tgz

AddLanguage da .dk
. snip

LanguagePriority en da nl et fr de el it ja kr no pl pt pt-br ltz ca es sv tw
ForceLanguagePriority Prefer Fallback

AddDefaultCharset ISO-8859-1

AddCharset ISO-8859-1 .iso8859-1 .latin1
. snip

AddType application/x-tar .tgz
#AddHandler cgi-script .cgi
#AddHandler send-as-is asis
AddHandler imap-file map
AddHandler type-map var
AddOutputFilter INCLUDES .shtml

#
Customizable error responses come in three flavors:
1) plain text 2) local redirects 3) external redirects
#
Some examples:
#ErrorDocument 500 "The server made a boo boo."
#ErrorDocument 404 /missing.html
#ErrorDocument 404 "/cgi-bin/missing_handler.pl"
#ErrorDocument 402 http://www.example.com/subscription_info.html

Alias /error/include/ "/your/include/path/"
Alias /error/ "/var/www/error/"
<IfModule mod_negotiation.c>
<IfModule mod_include.c>
 <Directory "/var/www/error">
 AllowOverride None
 Options IncludesNoExec
 AddOutputFilter Includes html
 AddHandler type-map var
 Order allow,deny
 Allow from all
 LanguagePriority en es de fr
 ForceLanguagePriority Prefer Fallback
 </Directory>

```

```

ErrorDocument 400 /error/HTTP_BAD_REQUEST.html.var
ErrorDocument 401 /error/HTTP_UNAUTHORIZED.html.var
ErrorDocument 403 /error/HTTP_FORBIDDEN.html.var
ErrorDocument 404 /error/HTTP_NOT_FOUND.html.var
ErrorDocument 405 /error/HTTP_METHOD_NOT_ALLOWED.html.var
ErrorDocument 408 /error/HTTP_REQUEST_TIME_OUT.html.var
ErrorDocument 410 /error/HTTP_GONE.html.var
ErrorDocument 411 /error/HTTP_LENGTH_REQUIRED.html.var
ErrorDocument 412 /error/HTTP_PRECONDITION_FAILED.html.var
ErrorDocument 413 /error/HTTP_REQUEST_ENTITY_TOO_LARGE.html.var
ErrorDocument 414 /error/HTTP_REQUEST_URI_TOO_LARGE.html.var
ErrorDocument 415 /error/HTTP_SERVICE_UNAVAILABLE.html.var
ErrorDocument 500 /error/HTTP_INTERNAL_SERVER_ERROR.html.var
ErrorDocument 501 /error/HTTP_NOT_IMPLEMENTED.html.var
ErrorDocument 502 /error/HTTP_BAD_GATEWAY.html.var
ErrorDocument 503 /error/HTTP_SERVICE_UNAVAILABLE.html.var
ErrorDocument 506 /error/HTTP_VARIANT_ALSO_VARIES.html.var
</IfModule>
</IfModule>

BrowserMatch "Mozilla/2" nokeepalive
BrowserMatch "MSIE 4\.\0b2;" nokeepalive downgrade-1.0 force-response-1.0
BrowserMatch "RealPlayer 4\.\0" force-response-1.0
BrowserMatch "Java/1\.\0" force-response-1.0
BrowserMatch "JDK/1\.\0" force-response-1.0

BrowserMatch "Microsoft Data Access Internet Publishing Provider" redirect-carefully
BrowserMatch "^WebDrive" redirect-carefully

#<Location /server-status>
SetHandler server-status
Order deny,allow
Deny from all
Allow from .your-domain.com
#</Location>

#<Location /server-info>
SetHandler server-info
Order deny,allow
Deny from all
Allow from .your-domain.com
#</Location>
Proxy Server directives. Uncomment the following lines to enable the proxy server:
<IfModule mod_proxy.c>
 ProxyRequests On

 <Proxy *>
 Order deny,allow
 Deny from all
 Allow from .your-domain.com
 </Proxy>

 Enable/disable the handling of HTTP/1.1 "Via:" headers.
 ("Full" adds the server version; "Block" removes all outgoing Via: headers)
 Set to one of: Off | On | Full | Block

 ProxyVia On

 To enable the cache as well, edit and uncomment the following lines:
 (no cacheing without CacheRoot)

 CacheRoot "/etc/httpd/proxy"
 CacheSize 5
 CacheGcInterval 4
 CacheMaxExpire 24
 CacheLastModifiedFactor 0.1
 CacheDefaultExpire 1
 NoCache a-domain.com another-domain.edu joes.garage-sale.com

</IfModule>
End of proxy directives.

```

## ### Section 3: Virtual Hosts

```

NameVirtualHost shiva.bom.labs.net
<VirtualHost shiva.bom.labs.net>
 ServerAdmin webmaster@shiva.bom.labs.net
 DocumentRoot /var/www/html
 ServerName shiva.bom.labs.net
 ErrorLog logs/shiva.bom.labs.net-error_log
 CustomLog logs/shiva.bom.labs.net-access_log common
</VirtualHost>
<VirtualHost www.bollywood.com>
 ServerAdmin webmaster@www.bollywood.com
 DocumentRoot /web/www.bollywood.com
 ServerName www.bollywood.com
 ErrorLog logs/www.bollywood.com
 CustomLog logs/www.bollywood.com-access_log common
</VirtualHost>
<VirtualHost www.hollywood.com>
 ServerAdmin webmaster@www.hollywood.com
 DocumentRoot /web/www.hollywood.com
 ServerName www.hollywood.com
 ErrorLog logs/www.hollywood.com
 CustomLog logs/www.hollywood.com-access_log common
</VirtualHost>
<VirtualHost linux.org>
 ServerAdmin webmaster@web/linux.org
 DocumentRoot /web/linux.org
 ServerName linux.org
 ErrorLog logs/www.hollywood.com
 CustomLog logs/linux.org-access_log common
</VirtualHost>

```

## SUMMARY

- /etc/httpd/modules/ link to /usr/lib/httpd/modules/\*.so

libphp4.so	mod_access.so	mod_actions.so	mod_alias.so
mod_asis.so	mod_auth_anon.so	mod_auth_dbm.so	mod_auth_digest.so
mod_auth_mysql.so	mod_auth_pgsql.so	mod_auth.so	mod_autoindex.so
mod_cern_meta.so	mod_cgi.so	mod_dav_fs.so	mod_dav.so
mod_deflate.so	mod_dir.so	mod_env.so	mod_expires.so
mod_headers.so	mod_imap.so	mod_include.so	mod_info.so
mod_log_config.so	mod_mime_magic.so	mod_mime.so	mod_negotiation.so
mod_perl.so	mod_proxy_connect.so	<b>mod_proxy_ftp.so</b>	<b>mod_proxy_http.so</b>
<b>mod_proxy.so</b>	mod_python.so	mod_rewrite.so	mod_setenvif.so
mod_speling.so	mod_ssl.so	mod_status.so	mod_suexec.so
mod_unique_id.so	mod_userdir.so	mod_usertrack.so	mod_vhost_alias.so

## Config Files

- /etc/httpd/conf/httpd.conf
- /var/www/html/
- /var/www/cgi-bin/
- /var/lock/httpd.lock
- /var/run/httpd.pid
- /var/run/httpd.scoreboard
- /var/log/httpd/access\_log
- /var/log/httpd/error\_log
- /etc/httpd/conf/srm.conf
- /etc/httpd/conf/access.conf
- /var/cache/httpd/
- conf.d/ \*.conf
- /etc/httpd/modules/ -> /usr/lib/httpd/modules/\*.so Hint: # **namei modules**
- /etc/httpd/manual/
- .htaccess
- .htpasswd
- etc/mime.types

## Commands

- `httpd -h`
- `httpd -t`
- `httpd -l`
- `httpd -L`
- `httpd -t`
- 
- 
- `htpasswd -c /usr/local/.htpasswd foo` Create the pwd file and add user. Done only once ever.
- `htpasswd /usr/local/.htpasswd foo` Change a user's pwd
- `htpasswd -n /usr/local/.htpasswd foo` Don't update file. Display results on stdout
- `htpasswd -m /usr/local/.htpasswd foo` Force MD5 encryption of the pwd
- `htpasswd -d /usr/local/.htpasswd foo` Force CRYPT encryption of the pwd [def]
- `htpasswd -s /usr/local/.htpasswd foo` Force SHA encryption of the pwd
- `htpasswd -b /usr/local/.htpasswd foo` Use pwd from cmd line rather than prompting for it
- `htpasswd -p /usr/local/.htpasswd foo` Do not encrypt the pwd. (plaintext)
- 
- `dbmmanage /usr/local/.http.passwd adduser foo`
- `dbmmanage /usr/local/.http.passwd check foo`
- `dbmmanage /usr/local/.http.passwd view foo`
- `dbmmanage /usr/local/.http.passwd update foo` Change foo's passwd
- `dbmmanage /usr/local/.http.passwd delete foo`
- 
- `htdbm -c -TDB /.htpasswd_dbm foo` The -c is given only the first time.
- `htdbm -m -TDB /.htpasswd_dbm foo` Force MD5 encryption of the passwd. [Deprecated]
- `htdbm -p -TDB /.htpasswd_dbm foo` Do not encrypt the pwd. Keep it plaintext
- `htdbm -s -TDB /.htpasswd_dbm foo` Force SHA encrypted of the pwd
- `htdbm -T -TDB /.htpasswd_dbm foo` DBM Type (SDBM|GDBM|DB|Default)
- `htdbm -l -TDB /.htpasswd_dbm foo` Displays the usernames from the database on stdout
- `htdbm -v -TDB /.htpasswd_dbm foo` Verify the username and password
- `htdbm -x -TDB /.htpasswd_dbm foo` Remove the username record from the DB

## Services

- service httpd restart

[illegible]